

Day 37 — Sun 11 Oct

Countdown: 1 day to CS Degree Paper-II + MCQ

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

● Degree — Paper-II revision (SWE)

Source: *_Shared/CORE_07_SoftwareEngineering.md*

CORE 07 — Software Engineering

Shared-core file 7 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points), **10/15-mark answer** (worked explanation + diagram).

0. Why this matters — where software engineering appears in your three papers

Elective	Where	How heavy
CS Degree	P-II §2 — "Concept of systems, Software development process modules, software project planning and management; cost estimation methods , scheduling, requirement analysis, software testing strategies , Quality Assurance ." Names cost estimation, scheduling and QA explicitly.	Heavy
CS Diploma	P-II §4 (System Analysis & Design) — "System concept: definition, characteristics, elements; Types: Physical, Abstract, Open, Closed; Information system; SDLC: different phases/stages , Role of the System Analyst, Factors affecting failure in SDLC."	Moderate–heavy
Computer Forensic	TP-II §3 — "SDLC: Steps, Waterfall model , Prototypes , Spiral model ; Software Metrics ; Software Project Management; Software Design (system, detailed, function-oriented, object-oriented, UI, design-level metrics); Coding and Testing (testing-level metrics); Software quality and reliability , clean-room approach, software reengineering , reverse engineering ." The most detailed of the three.	Heavy

This is a strong shared topic and a "safe" one — it is descriptive, diagram-friendly and rewards structured answers, so it yields Section-A marks in every paper. The two numerical islands — **COCOMO / function points** and **PERT-CPM / cyclomatic complexity** — are the highest-return practice, because worked numericals are unambiguous to mark. Study the process models, testing and cost estimation deeply; they recur across all three electives.

Topic checklist

- [] System concept, characteristics, elements; types (physical/abstract, open/closed)
- [] Software, its characteristics; the **software crisis**
- [] **SDLC phases** in order
- [] **Process models**: Waterfall, Prototype, Spiral, Incremental, Iterative, RAD, V-model, Agile — each with a diagram, when-to-use, pros and cons
- [] **SRS** — contents, characteristics of a good SRS
- [] Feasibility study (TELOS)
- [] Project planning; role of the system analyst; failure factors

- [] **Cost estimation — COCOMO basic and intermediate (worked); function points (worked)**
- [] Scheduling: **Gantt charts, PERT/CPM with a worked critical path**
- [] Design: function- vs object-oriented, modularity, abstraction
- [] **Coupling and cohesion — every type ranked, with examples**
- [] User-interface design principles
- [] **Testing**: unit, integration (top-down/bottom-up/sandwich), system, acceptance
- [] **Black-box**: equivalence partitioning, BVA, cause-effect, decision tables
- [] **White-box**: statement/branch/path coverage, **cyclomatic complexity (worked)**
- [] Alpha/beta/regression testing; **V&V**
- [] Software metrics (LOC, FP, Halstead)
- [] SQA; quality models — **McCall, ISO 9126, CMM levels 1–5**
- [] Software reliability (MTBF, MTTF, MTTR)
- [] Maintenance types; reengineering and reverse engineering

1. Systems, software and the software crisis

Concept — what a system is (Diploma §4)

A **system** is an **organised set of interrelated components that work together toward a common goal**, transforming **inputs** into **outputs**. Every system has:

- **Inputs** (data/resources), **processing** (the transformation), **outputs** (results).
- **Boundary** (what is inside vs outside), **environment** (what surrounds it), **interfaces** (connections between components/other systems).
- **Feedback and control** (comparing output with a goal and adjusting).

Characteristics of a system: organisation, interaction, interdependence, integration, and a central objective.

Types of system (a Diploma 5-marker):

Pair	Distinction
Physical vs abstract	Physical = tangible (computers, people); abstract = conceptual (a formula, a model)
Open vs closed	Open = exchanges with its environment (adapts); closed = isolated, self-contained (no environment interaction)
Deterministic vs probabilistic	Deterministic = predictable behaviour (a program); probabilistic = uncertain (weather, inventory demand)
Manual vs automated	Done by people vs by machines

An **information system** is a system that collects, processes, stores and distributes **information** to support decision-making and control (e.g. TPS, MIS, DSS, ESS).

Software and its characteristics

Software is not just code: it is **programs + data + documentation** — the set of instructions and associated artefacts that make a computer do useful work. Its **characteristics** distinguish it from hardware:

- Software is **developed/engineered, not manufactured** — cost is in design, not production.
- Software **does not wear out** — but it **deteriorates** through repeated changes (the "bathtub curve" for hardware rises again at end-of-life; software's failure rate would be flat except that each change introduces new defects — an "**increasing failure rate due to changes**").
- Most software is **custom-built**, not assembled from standard components (though reuse is

rising).

- Software is **intangible and complex**.

Types: system software (OS, compilers), application software, engineering/scientific, embedded, product-line, web, AI software.

The software crisis

The **software crisis** (term coined at the 1968 NATO conference) is the historical realisation that software projects routinely came in **over budget, over time, of poor quality, hard to maintain, and often failed outright**, because ad-hoc "code-and-fix" development could not scale to large systems. Its symptoms are the standard 5-mark list:

- Projects **exceeded budget and schedule**.
- Software was of **poor quality / unreliable**.
- Software was **difficult to maintain and did not meet requirements**.
- Productivity could not keep up with demand.

Software engineering is the disciplined response: *"the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software"* (IEEE). It brings engineering principles — process, measurement, quality control — to software.

Likely exam questions

- **[5]** Define a system. State its characteristics and elements.
- **[5]** Differentiate between open and closed / physical and abstract systems.
- **[5]** What is the software crisis? State its causes and symptoms.
- **[5]** What are the characteristics of software? How does it differ from hardware?
- **[10]** What is software engineering? Explain the software crisis that led to it and the characteristics of software.

MCQ traps

Trap	Correct answer
"Software wears out?"	No — it deteriorates (due to changes), it does not wear out.
"Software = ?"	Programs + data + documentation , not just code.
"A system that interacts with its environment is?"	Open .
"The term 'software crisis' arose at?"	The 1968 NATO conference .
"Software is manufactured?"	No — engineered/developed .

2. The Software Development Life Cycle (SDLC)

Concept

The **SDLC** is the structured sequence of **phases** a software project passes through from conception to retirement. The *process models* in §3 are different **arrangements** of these same phases; learn the phases once and every model becomes a rearrangement of them.

#	Phase	What happens	Output artefact
1	Requirement gathering & analysis	Elicit, analyse and document what the system must do	SRS document

2	Feasibility study (often merged with 1)	Check technical/economic/legal viability	Feasibility report
3	Design	Architecture, modules, data, interfaces (HLD then LLD)	Design document (SDD)
4	Coding / implementation	Translate design into source code	Source code
5	Testing	Verify and validate — find and fix defects	Test reports, working software
6	Deployment / installation	Release to the user environment	Deployed system
7	Maintenance	Correct, adapt, enhance after delivery	Updated system

The maintenance phase is the longest and most expensive — commonly **60–70% of total lifetime cost**. That single fact is worth a mark and reframes the whole subject: the point of good engineering is to reduce maintenance cost.

Role of the system analyst (Diploma): the bridge between users and developers — studies the existing system, elicits and analyses requirements, prepares the SRS, designs the logical solution, and guides implementation. Skills: analytical, communication, technical, and domain knowledge.

Factors causing SDLC failure (Diploma): unclear/changing requirements, poor planning and estimation, inadequate user involvement, weak project management, unrealistic schedules, insufficient testing, and poor communication.

Likely exam questions

- **[5]** What is the SDLC? List its phases in order.
- **[5]** Explain the role of the system analyst.
- **[5]** State the factors that cause failure in the SDLC.
- **[10]** Explain the phases of the SDLC with the activities and output of each phase.

MCQ traps

Trap	Correct answer
"First phase of the SDLC?"	Requirement gathering & analysis .
"Most expensive / longest phase?"	Maintenance .
"Output of the requirement phase?"	The SRS .
"HLD and LLD belong to which phase?"	Design .

3. Software process models — compared

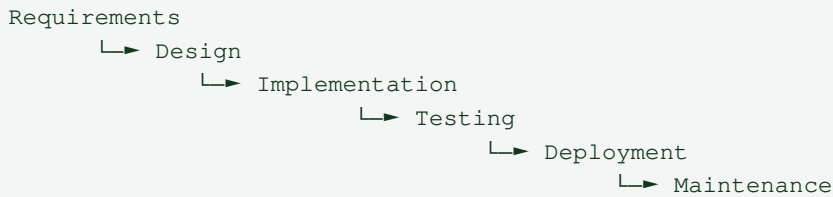
Concept

A **process (life-cycle) model** prescribes the **order and manner** in which SDLC phases are carried out. There is no universally best model; the choice depends on **requirement clarity, project size, risk, and how much the customer needs to be involved**. This comparison is the single most examined SE topic — expect a 10- or 15-marker asking you to explain several models with diagrams, pros/cons and when to use each.

3.1 Waterfall model

Idea: phases flow **strictly downward**, each completed and signed off before the next begins — like water cascading. No going back (in the pure form).

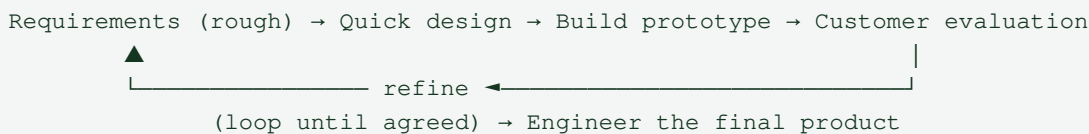
Diagram to draw: a staircase of boxes descending left-to-right — Requirements → Design → Implementation → Testing → Deployment → Maintenance — each feeding the next.



When to use	Pros	Cons
Requirements are clear, fixed and well understood; short, small projects; mature domains	Simple, easy to manage; clear milestones and documentation; disciplined	No working software until late; very poor at handling changing requirements; risk found late; customer sees the product only at the end

3.2 Prototype model

Idea: build a **quick, throwaway (or evolutionary) working model** early, show it to the customer, gather feedback, and **refine iteratively** until requirements are clear — *then* build the real system. Best when **requirements are unclear or the customer cannot articulate them**.



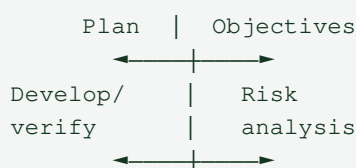
When to use	Pros	Cons
Requirements unclear/evolving; user needs early visualisation; new/experimental domains	Early user feedback; reduces requirement risk; users see something concrete quickly	Throwaway effort; customer may mistake the prototype for the product; poor documentation ; can invite scope creep

3.3 Spiral model (Boehm, 1986)

Idea: a **risk-driven** model that combines iterative prototyping with the systematic waterfall. The project spirals outward through repeated cycles, each with **four quadrants**:

1. **Determine objectives**, alternatives, constraints.
2. **Identify and resolve risks** (prototyping, analysis) — *the defining feature*.
3. **Develop and verify** the next-level product.
4. **Plan** the next iteration.

Each loop produces a more complete version; **the radius represents cost, the angle represents progress**.



When to use	Pros	Cons

Large, expensive, high-risk projects ; requirements evolve; R&D	Explicit risk management ; flexible; supports change; early prototypes	Costly and complex ; needs risk-assessment expertise ; overkill for small projects; hard to estimate
--	---	--

3.4 Incremental model

Idea: build the system in **increments** — deliver a working core first, then add functionality in successive, deliverable pieces. Each increment goes through its own design-code-test mini-cycle. **The customer gets usable software early** and each increment adds capability.

When to use	Pros	Cons
Core requirements clear, extras can follow; need early delivery	Early working product ; easier to test/debug per increment; lower initial delivery risk	Needs good architecture up front; total cost may be higher; integration overhead

3.5 Iterative (enhancement) model

Idea: start with a **simple implementation of a subset** of requirements and **iteratively enhance** it, **refining the same product** version after version until complete. (Distinction from incremental: incremental **adds new pieces**; iterative **refines the whole** repeatedly. In practice most modern methods are **iterative and incremental**.)

When to use	Pros	Cons
Requirements understood but large; want early feedback	Feedback each cycle; changes accommodated; early risk visibility	Re-work each iteration; needs more resources/management

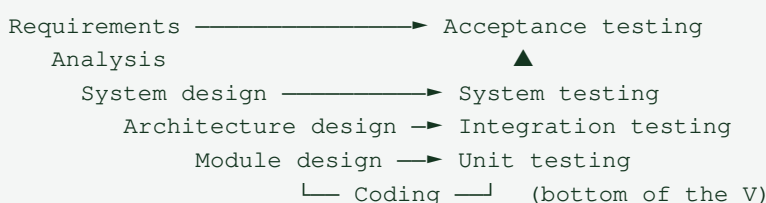
3.6 RAD — Rapid Application Development

Idea: compress the schedule by using **component reuse, powerful CASE/4GL tools, and parallel development by multiple teams**, each building a component in a short **time-box** (60–90 days), then integrating. Heavy user involvement via workshops (JAD).

When to use	Pros	Cons
Requirements reasonably known; tight deadline ; reusable components exist; modular system	Very fast delivery ; reuse; strong user involvement	Needs skilled developers and strong tooling ; needs a modular, componentisable system; not for large, complex or high-performance systems

3.7 V-model (Verification and Validation model)

Idea: an extension of waterfall where **each development phase has a corresponding testing phase**, drawn as a **V**. The **left arm** goes down through the development phases; the **right arm** rises through the matching test phases; the two arms are linked so that test plans are written **during** the corresponding development phase — **testing is planned in parallel with development**, not deferred.



When to use	Pros	Cons
-------------	------	------

Requirements clear and fixed; small/medium projects where quality is critical (medical, defence)	Testing planned early ; each phase verified; disciplined, high quality	Rigid like waterfall; poor at change ; no early prototype
--	---	--

3.8 Agile

Idea: deliver working software in short **iterations (sprints, ~2–4 weeks)**, embracing **changing requirements**, continuous customer collaboration, and self-organising teams. Guided by the **Agile Manifesto (2001)**: *individuals and interactions over processes and tools; working software over documentation; customer collaboration over contract negotiation; responding to change over following a plan*. Frameworks: **Scrum** (sprints, product/sprint backlog, daily stand-up, roles: Product Owner, Scrum Master, Team), **XP** (pair programming, TDD, continuous integration), **Kanban**.

When to use	Pros	Cons
Requirements volatile ; need frequent delivery; engaged customer; small–medium teams	Adapts to change ; continuous delivery; high customer satisfaction; early ROI	Less documentation ; hard to scale to very large teams; needs experienced, disciplined teams; unpredictable final scope/cost

The master comparison table — reproduce this

Model	Requirements	Risk handling	Customer involvement	Working software	Best for
Waterfall	Fixed, clear	Poor (late)	Low (start & end)	Late	Small, well-understood projects
Prototype	Unclear	Medium	High	Early (prototype)	Vague requirements
Spiral	Evolving	Excellent (risk-driven)	Medium	Each spiral	Large, high-risk projects
Incremental	Core clear	Good	Medium–high	Early, in increments	Need early partial delivery
Iterative	Understood, large	Good	Medium	Each iteration	Large systems, evolving detail
RAD	Known, modular	Medium	High (workshops)	Very fast	Tight deadline, reusable parts
V-model	Fixed, clear	Medium	Low	Late	Quality-critical, stable reqs
Agile	Volatile	Good (continuous)	Very high	Continuous	Changing requirements

Likely exam questions

- **[5]** Explain the Waterfall model with a diagram, stating two advantages and two drawbacks.
- **[5]** When is the prototype model preferred over the waterfall model?
- **[5]** Explain the spiral model. Why is it called risk-driven?
- **[5]** Differentiate between the incremental and iterative models.
- **[5]** Explain the V-model. How does it improve on the waterfall model?
- **[5]** State the four values of the Agile Manifesto.

- **[10]** Compare the Waterfall, Prototype and Spiral models with diagrams, pros, cons and when to use each.
- **[15]** Explain the various software process models (Waterfall, Prototype, Spiral, Incremental, RAD, V-model, Agile) with diagrams, and recommend a suitable model for (a) a well-understood payroll system, (b) a research project with unclear requirements, and (c) a large high-risk banking system, justifying each choice.

MCQ traps

Trap	Correct answer
"Which model is 'risk-driven'?"	Spiral (Boehm, 1986).
"Which model has no working software until the end?"	Waterfall .
"Which model pairs each dev phase with a test phase?"	V-model .
"Which model builds a throwaway model first?"	Prototype .
"Which best handles changing requirements?"	Agile .
"RAD relies on?"	Reusable components and CASE/4GL tools .
"Incremental vs iterative — incremental?"	Adds new features ; iterative refines the whole.
"Scrum roles?"	Product Owner, Scrum Master, Team .

4. Software Requirements Specification (SRS)

Concept

The **SRS** is the official document that records **what the system must do** (functional requirements) and **how well** (non-functional requirements). It is the **contract** between customer and developer and the baseline against which the finished system is validated.

Requirement types:

- **Functional requirements** — specific behaviours/services ("the system shall generate a monthly report").
- **Non-functional requirements** — quality attributes and constraints: performance, security, usability, reliability, portability, scalability (the "-ilities").
- **Domain requirements** — arising from the application domain.

Characteristics of a good SRS (a guaranteed 5-marker — the "-ct" list):

Quality	Meaning
Correct	Every requirement truly reflects a need
Complete	All requirements included; nothing missing
Unambiguous	Each requirement has exactly one interpretation
Consistent	No requirement contradicts another
Verifiable / testable	You can devise a test to check it is met
Modifiable	Well-structured so changes are easy

Traceable	Each requirement can be traced to its origin and to design/test
Ranked	Prioritised by importance/stability
Feasible	Achievable within constraints

IEEE 830 is the standard SRS template. The costliest defects are requirement defects — a requirement error found in maintenance can cost ~**100×** what it costs to fix during the requirement phase; this is the argument for investing in a good SRS.

Likely exam questions

- **[5]** What is an SRS? Differentiate functional and non-functional requirements.
- **[5]** State the characteristics of a good SRS.
- **[10]** What is requirement analysis? Explain the contents and desirable characteristics of an SRS document.

MCQ traps

Trap	Correct answer
"The system shall respond in 2 seconds' is a?"	Non-functional requirement.
"The SRS is produced in which phase?"	Requirement analysis .
"Standard for SRS?"	IEEE 830 .
"An SRS must be unambiguous means?"	One interpretation only .

5. Feasibility study

Concept

The **feasibility study** decides *whether the project is worth doing* before serious resources are committed. The classic checklist is **TELOS**:

Feasibility	Question
Technical	Can it be built with available technology and skills?
Economic	Do the benefits outweigh the costs? (cost-benefit analysis , ROI, payback)
Legal	Does it comply with laws, licences, contracts, data-protection rules?
Operational	Will it work within the organisation and be accepted by users?
Schedule (time)	Can it be delivered within the required timeframe?

Economic feasibility is usually decisive; it uses cost-benefit analysis, **payback period**, **net present value (NPV)** and **return on investment (ROI)**.

Likely exam questions

- **[5]** What is a feasibility study? Explain its types (TELOS).
- **[5]** Explain economic feasibility and cost-benefit analysis.

MCQ traps

Trap	Correct answer
"TELOS stands for?"	Technical, Economic, Legal, Operational, Schedule.
"Cost-benefit analysis is part of?"	Economic feasibility.
"'Will users accept it?' is which feasibility?"	Operational.

6. Project planning and management

Concept

Software project management balances the **project management triangle** — **scope, time, cost** (with quality at the centre). Core planning activities:

- **Scope definition** and **work breakdown structure (WBS)** — decompose the project into tasks.
- **Estimation** — size, effort, cost, time (see §7).
- **Scheduling** — sequence tasks, assign resources (see §8).
- **Risk management** — identify, analyse, prioritise, plan mitigation, monitor.
- **Staffing / resource planning; configuration management** (version control, change control); **monitoring and control**.

Putnam's / Brooks's insight: *"Adding manpower to a late software project makes it later"* (**Brooks's Law**) — because of the communication overhead among $n(n-1)/2$ pairs and ramp-up time. Effort and time are not freely interchangeable.

Likely exam questions

- **[5]** What is the project management triangle? Explain scope, time and cost.
- **[5]** State Brooks's Law and explain why it holds.
- **[5]** What is a work breakdown structure (WBS)?
- **[10]** Explain the activities involved in software project planning and management.

MCQ traps

Trap	Correct answer
"'Adding people to a late project makes it later' is?"	Brooks's Law.
"The project triangle constrains?"	Scope, time, cost.
"Decomposing a project into tasks gives a?"	WBS.

7. Cost estimation — COCOMO and function points

This is a top numerical topic. Work every formula by hand and show every step.

Concept

Estimation predicts the **size**, then the **effort** (person-months), then the **time** and **cost** of a project. Size is measured in **LOC (lines of code)** or **function points (FP)**.

The two headline techniques are **COCOMO** (from size) and **Function Point Analysis** (from functionality).

COCOMO — Constructive Cost Model (Boehm, 1981)

COCOMO estimates **effort** and **development time** from the estimated size in **KLOC** (thousands of delivered lines of code). It comes in three project **modes**:

Mode	Nature	Team/size	Example
Organic	Small, simple; experienced team; familiar, stable environment	Small (< ~50 KLOC)	A simple inventory/payroll system
Semi-detached	Medium size and complexity; mixed experience	Medium (~50–300 KLOC)	A DBMS, a compiler, a transaction system
Embedded	Large, complex; tight hardware/software/regulatory constraints	Large (> ~300 KLOC)	Real-time OS, avionics, ATC

Basic COCOMO

Two formulas:

Effort $E = a \times (\text{KLOC})^b$ person-months (PM)

Time $D = c \times (E)^d$ months

Persons required $P = E / D$; productivity = KLOC / E.

Coefficients (**memorise the table** — it is handed out rarely, so learn it):

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Worked example 1 — Basic COCOMO, organic

A project is estimated at 32 KLOC, organic mode. Find effort, development time, number of people and productivity.

Effort $E = 2.4 \times (32)^{1.05}$

$32^{1.05} = 32 \times 32^{0.05}$. $32^{0.05} = e^{(0.05 \cdot \ln 32)} = e^{(0.05 \times 3.4657)} = e^{0.17329} = 1.1892$.

So $32^{1.05} = 32 \times 1.1892 = 38.05$.

$E = 2.4 \times 38.05 = 91.3 \approx 91$ person-months.

Time $D = 2.5 \times (91.3)^{0.38}$

$91.3^{0.38} = e^{(0.38 \cdot \ln 91.3)} = e^{(0.38 \times 4.514)} = e^{1.7153} = 5.559$.

$D = 2.5 \times 5.559 = 13.9 \approx 14$ months.

People $P = E / D = 91.3 / 13.9 = 6.6 \approx 7$ persons.

Productivity = $32 / 91.3 = 0.35$ KLOC per person-month (≈ 350 LOC/PM).

Worked example 2 — Basic COCOMO, embedded

Size = 400 KLOC, embedded mode.

$E = 3.6 \times (400)^{1.20}$. $400^{1.20} = e^{(1.2 \cdot \ln 400)} = e^{(1.2 \times 5.9915)} = e^{7.1898} = 1329$.

$E = 3.6 \times 1329 = 4784 \approx 4785$ person-months.

$D = 2.5 \times (4784)^{0.32} = 2.5 \times e^{(0.32 \cdot \ln 4784)} = 2.5 \times e^{(0.32 \times 8.473)} = 2.5 \times e^{2.7113} =$

$2.5 \times 15.05 = 37.6 \approx 38$ months. $P = 4784/37.6 \approx 127$ persons.

Intermediate COCOMO

Basic COCOMO ignores project-specific factors. **Intermediate COCOMO** multiplies the nominal effort by an **Effort Adjustment Factor (EAF)** — the product of **15 cost drivers** rated from *very low* to *extra high*, grouped as **product, hardware, personnel and project** attributes (e.g. required reliability RELY, database size DATA, complexity CPLX, analyst capability ACAP, programmer capability PCAP, use of tools TOOL, schedule SCED).

$$E = a \times (KLOC)^b \times EAF$$

with intermediate coefficients:

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

EAF = product of the 15 cost-driver multipliers; each rating maps to a number (nominal = 1.0, higher reliability > 1, better staff < 1). EAF typically ranges ~0.9 to ~1.4.

Worked example 3 — Intermediate COCOMO

Organic, 32 KLOC. Suppose the selected cost drivers give multipliers whose product is $EAF = 1.15$.

Nominal $E = 3.2 \times (32)^{1.05} = 3.2 \times 38.05 = 121.8$.

Adjusted $E = 121.8 \times 1.15 = 140.0$ person-months.

$D = 2.5 \times (140)^{0.38} = 2.5 \times e^{(0.38 \times 4.942)} = 2.5 \times e^{1.878} = 2.5 \times 6.54 = 16.4 \approx 16$ months.

(**Detailed COCOMO** goes further, applying cost drivers phase by phase — mention it exists.)

Function Point Analysis (FPA)

Problem COCOMO has: LOC is **language-dependent** and unknown until late. **Function points** measure the **size from the functionality delivered to the user**, independent of language, and can be computed **early from the requirements**.

Five function types, each counted and weighted by complexity (simple/average/complex):

Component	Meaning	Simple	Avg	Complex
EI — External Inputs	User inputs that update data (forms)	3	4	6
EO — External Outputs	Outputs to the user (reports, messages)	4	5	7
EQ — External Inquiries	Input–output queries (no update)	3	4	6
ILF — Internal Logical Files	Internal data groups maintained	7	10	15
EIF — External Interface Files	Data used but maintained elsewhere	5	7	10

Steps:

- UFP (Unadjusted Function Points)** = Σ (count $\times$ weight) over all five types.
- VAF (Value Adjustment Factor)** = $0.65 + 0.01 \times \Sigma Fi$, where the **14 general system characteristics** are each rated 0–5 (ΣFi ranges 0–70, so VAF ranges **0.65 to 1.35**).
- FP = UFP $\times$ VAF.**

4. Optionally, **LOC = FP × (LOC per FP for the language)** to feed COCOMO.

Worked example 4 — Function points

A system has: EI = 22 (avg), EO = 20 (avg), EQ = 10 (simple), ILF = 4 (avg), EIF = 2 (avg).

Total degree of influence $\sum Fi = 30$.

Step 1 — UFP (use the weights above):

Type	Count	Weight	Product
EI (avg)	22	4	88
EO (avg)	20	5	100
EQ (simple)	10	3	30
ILF (avg)	4	10	40
EIF (avg)	2	7	14
		UFP	272

Step 2 — VAF = $0.65 + 0.01 \times 30 = 0.65 + 0.30 = \mathbf{0.95}$.

Step 3 — FP = $272 \times 0.95 = \mathbf{258.4 \approx 258}$ function points.

Step 4 — to LOC (say the language averages 50 LOC/FP):

LOC = $258 \times 50 = 12\,900 \approx \mathbf{12.9\ KLOC}$, which could then be fed into COCOMO.

Likely exam questions

- **[5]** Explain the three modes of COCOMO with examples.
- **[5]** State the basic COCOMO formulas and their coefficient table.
- **[5]** Differentiate basic and intermediate COCOMO. What is the EAF?
- **[5]** What are function points? List the five function types.
- **[10]** For an organic project of 40 KLOC, compute the effort, development time and number of people using basic COCOMO. ($E = 2.4 \times 40^{1.05} = 2.4 \times 46.9 = 112.6\ PM$; $D = 2.5 \times 112.6^{0.38} = 2.5 \times 5.98 = 15.0\ months$; $P \approx 8$.)
- **[10]** Compute the function-point count for the given system and convert it to LOC.
- **[15]** Explain software cost estimation. Describe COCOMO (basic and intermediate) and Function Point Analysis, and solve one numerical of each.

MCQ traps

Trap	Correct answer
"COCOMO estimates from?"	KLOC (size) .
"Organic basic coefficients a, b?"	2.4 and 1.05 .
"Which mode has the highest exponent b?"	Embedded (1.20) .
"Intermediate COCOMO adds the?"	EAF (from 15 cost drivers) .
"Function points depend on the programming language?"	No — they are language-independent.
"How many function-point types?"	Five (EI, EO, EQ, ILF, EIF).

"VAF range?"	0.65 to 1.35.
"Who developed COCOMO?"	Barry Boehm (1981).

8. Project scheduling — Gantt, PERT and CPM

Concept

Scheduling sequences the estimated tasks over calendar time and assigns resources. Two visual tools dominate the exam: the **Gantt chart** (simple bar timeline) and the **network diagram** analysed with **PERT/CPM** (dependencies and the critical path).

Gantt chart

A **horizontal bar chart**: each task is a bar whose **length = duration** and **position = start and end dates**; bars can overlap to show parallel work, and **milestones** are marked as diamonds. It shows **what runs when** and **progress** at a glance, but **does not clearly show task dependencies** — that is what PERT/CPM adds.

PERT and CPM

Both use an **activity network** (nodes and arrows) to find the **critical path** — the longest path through the project, which **determines the minimum project duration**. Any delay on a critical activity delays the whole project.

	CPM (Critical Path Method)	PERT (Program Evaluation & Review Technique)
Time estimate	Single, deterministic duration	Three estimates → probabilistic
Focus	Time–cost trade-off (crashing)	Uncertainty / risk in scheduling
Suited to	Repetitive, well-known projects (construction)	Research/new projects with uncertain times
Activities	Activity-on-node commonly	Activity-on-arrow commonly

PERT expected time for an activity from optimistic (o), most likely (m), pessimistic (p):

$$t_e = (o + 4m + p) / 6, \text{ variance } \sigma^2 = ((p - o)/6)^2$$

Scheduling terms: **ES** (earliest start), **EF** (earliest finish = ES + duration), **LS** (latest start), **LF** (latest finish). **Slack/float** = **LS – ES** = **LF – EF**. **Critical activities have zero slack**. A **forward pass** computes ES/EF; a **backward pass** computes LS/LF.

Worked example — critical path

Activities, durations (weeks) and predecessors:

Activity	Duration	Predecessor
A	3	—
B	4	A
C	2	A

D	5	B
E	3	C
F	2	D, E

Forward pass (ES, EF):

- A: ES 0, EF 3.
- B: ES 3, EF 7. C: ES 3, EF 5.
- D: ES 7, EF 12. E: ES 5, EF 8.
- F: ES = max(EF of D, E) = max(12, 8) = 12, EF = 14.

Project duration = 14 weeks.

Paths and lengths:

- A→B→D→F = 3+4+5+2 = **14** ← longest
- A→C→E→F = 3+2+3+2 = 10

Critical path = A → B → D → F (14 weeks). Activities C and E have slack: E's slack = 12 – 8 = **4 weeks**, so E can slip up to 4 weeks without delaying the project. C and E are **non-critical**.

If asked to "crash": to shorten the project you must shorten a **critical** activity (A, B, D or F); crashing C or E achieves nothing until the critical path changes.

Likely exam questions

- **[5]** Differentiate between a Gantt chart and a PERT chart.
- **[5]** Differentiate between PERT and CPM.
- **[5]** Define critical path, slack/float, ES, EF, LS, LF.
- **[5]** State the PERT expected-time formula and compute t_e for $o=2$, $m=4$, $p=12$. ($t_e = (2+16+12)/6 = 5$.)
- **[10]** For the given activity table, draw the network, compute ES/EF/LS/LF, find the critical path and the project duration, and state the slack of each activity.

MCQ traps

Trap	Correct answer
"The critical path is the?"	Longest path; it sets the minimum project duration.
"Slack on a critical activity?"	Zero .
"PERT uses how many time estimates?"	Three (o, m, p).
"PERT expected time formula?"	(o + 4m + p)/6 .
"Which chart shows dependencies best?"	PERT/CPM network (not Gantt).
"CPM emphasises?"	Time–cost trade-off .

9. Software design

Concept

Design transforms the *what* of the SRS into the *how* — the blueprint for coding. It has two

levels: **High-Level Design (HLD / architectural)** — modules and their relationships — and **Low-Level Design (LLD / detailed)** — the internal logic of each module. Good design rests on **abstraction, modularity, information hiding** and low coupling / high cohesion.

Fundamental design concepts:

Concept	Meaning
Abstraction	Focus on essential features, hide detail (procedural, data, control abstraction)
Modularity	Divide the system into separately named, addressable modules
Information hiding	Each module hides its internal data/algorithm behind an interface
Stepwise refinement	Top-down elaboration of detail
Refactoring	Improving internal structure without changing external behaviour

Modularity is desirable because it makes a system easier to understand, test and maintain — but *too many* tiny modules raise integration cost, and *too few* large ones are hard to understand, so there is an **optimum number of modules** minimising total cost.

Function-oriented vs object-oriented design

	Function-oriented (procedural)	Object-oriented
Decomposition	By functions/processes (top-down)	By objects = data + behaviour
Central abstraction	The function : data is separate and shared	The object/class : data and functions bound together (encapsulation)
Data	Often global/shared, passed around	Encapsulated , hidden inside objects
Tools	DFD , structure charts, data dictionary	Class/object diagrams, UML
Key ideas	Modules, coupling/cohesion	Encapsulation, inheritance, polymorphism, abstraction
Change impact	A data-structure change can ripple widely	Localised behind object interfaces
Suits	Well-defined procedural problems	Large, evolving systems; reuse

The **DFD (Data Flow Diagram)** is the core function-oriented tool: circles = processes, arrows = data flows, open rectangles = data stores, squares = external entities; **Level 0 = context diagram**, refined into Levels 1, 2, ...

Coupling and cohesion — ranked, with examples

The **two central quality measures of a modular design**, and a guaranteed exam question. The design goal is **low coupling and high cohesion**.

Coupling = the **degree of interdependence *between* modules**. **Low (loose) coupling is better** — modules can be changed, tested and reused independently. Ranked from **best (lowest)** to **worst (highest)**:

Rank	Coupling type	Meaning	Example

1 (best)	Data coupling	Modules share data only through simple parameters (atomic values)	<code>computeTax(income)</code> returns a value
2	Stamp coupling	A whole data structure is passed but only part is used	Passing an entire <code>Employee</code> record to get the salary
3	Control coupling	One module passes a control flag that dictates the other's logic	<code>printReport(flag)</code> where flag chooses the format
4	External coupling	Modules share an externally imposed format/protocol/device	Both tied to a specific file format or hardware
5	Common coupling	Modules share global data	Several modules read/write the same global variable
6 (worst)	Content coupling	One module directly accesses or modifies another's internals	Module A jumps into or changes Module B's local data

Cohesion = the **degree to which the elements *within* a single module belong together**.

High cohesion is better — a module should do **one well-defined thing**. Ranked from **worst (lowest)** to **best (highest)**:

Rank	Cohesion type	Meaning	Example
1 (worst)	Coincidental	Elements grouped arbitrarily , no relationship	A "utilities" module of unrelated functions
2	Logical	Elements do similar kinds of things, chosen by a flag	One module doing "all input", selected by a parameter
3	Temporal	Elements grouped because they run at the same time	An <code>init()</code> that opens files, zeroes counters, sets flags
4	Procedural	Elements follow a sequence of control	Steps executed in order but on unrelated data
5	Communicational	Elements operate on the same data	Functions all working on the same input record
6	Sequential	Output of one element is the input of the next	Read → validate → format the same data in a chain
7 (best)	Functional	All elements contribute to one single, well-defined task	<code>computeSquareRoot(x)</code>

The one-line goal to state: *strive for high (functional) cohesion and low (data)*

coupling. Memory aids for the orders: coupling best→worst = **Data, Stamp, Control, External, Common, Content**; cohesion worst→best = **Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional**.

User-interface (UI) design

Golden rules (Shneiderman / Mandel):

1. **Place the user in control** — flexible, forgiving, interruptible.
2. **Reduce the user's memory load** — recognition over recall, meaningful defaults.
3. **Make the interface consistent** — uniform look, layout and behaviour.

Add principles: **feedback** for every action, **error prevention and easy recovery** (undo), **simplicity/aesthetics**, and **accessibility**. The UI design process is **iterative**: analyse users/tasks → design → prototype → **evaluate with users** → refine.

Likely exam questions

- **[5]** Differentiate between coupling and cohesion. State the design goal.
- **[5]** List the types of coupling from best to worst with an example of each.
- **[5]** List the types of cohesion from worst to best with an example of each.
- **[5]** Differentiate function-oriented and object-oriented design.
- **[5]** State the golden rules of user-interface design.
- **[10]** Explain coupling and cohesion in detail, ranking every type with examples, and state why low coupling and high cohesion are desirable.
- **[10]** Explain the fundamental concepts of software design: abstraction, modularity and information hiding, and the DFD as a design tool.

MCQ traps

Trap	Correct answer
"Design goal for coupling and cohesion?"	Low coupling, high cohesion .
"Best (lowest) coupling?"	Data coupling . Worst = content coupling .
"Best (highest) cohesion?"	Functional cohesion . Worst = coincidental .
"Modules sharing global data have?"	Common coupling .
"'Output of one is input of the next' is which cohesion?"	Sequential .
"Passing a whole record to use one field is?"	Stamp coupling .
"Binding data and functions together is?"	Encapsulation (OO) .
"A DFD process is drawn as a?"	Circle/bubble .

10. Software testing

Concept

Testing is the process of **executing a program with the intent of finding errors** (Myers). It can only show the **presence** of defects, never their absence (Dijkstra). Vocabulary to keep straight: an **error** (human mistake) causes a **fault/defect/bug** (in the code) which, when executed, causes a **failure** (wrong behaviour).

Levels of testing

Level	What is tested	By whom	Notes
Unit testing	A single module/function in isolation	Developer	Needs stubs (dummy called modules) and drivers (dummy calling modules)

Integration testing	Interactions between combined modules	Developers/ testers	Strategies below
System testing	The complete, integrated system against requirements	Independent testers	Functional + non-functional (performance, security, load, stress, usability)
Acceptance testing	The system against user needs, for sign-off	Customer/user	Alpha and beta (below)

Integration strategies — a favourite comparison:

Strategy	How	Needs	Pros / cons
Top-down	Integrate from the top module downward , replacing lower modules with stubs	Stubs	Major control logic tested early; but lower-level modules tested late, many stubs needed
Bottom-up	Integrate from the lowest modules upward , using drivers	Drivers	Low-level modules tested thoroughly early; but the whole system emerges only at the end, no early prototype
Sandwich (hybrid)	Combine top-down and bottom-up around a target layer	Both stubs and drivers	Balances the two; complex to plan
Big-bang	Integrate everything at once and test	—	Simple but fault localisation is very hard — avoid for large systems

Black-box testing (functional)

Tests **behaviour against the specification**, with **no knowledge of internal code** — inputs in, outputs checked. Techniques:

Technique	Idea	Example
Equivalence Partitioning (EP)	Divide inputs into classes that should behave the same ; test one value per class (one valid, plus invalid classes)	Age 18–60 valid → test one value in 18–60, one < 18, one > 60
Boundary Value Analysis (BVA)	Bugs cluster at boundaries — test at, just below and just above each edge	For 18–60: test 17, 18, 19 ... 59, 60, 61
Cause-Effect Graphing	Map input causes to output effects as a logic graph, then derive test cases	Complements EP for combinations
Decision Table testing	Tabulate combinations of conditions → actions ; test each rule	See below
State transition testing	Test valid/invalid state changes	Login → locked after 3 failures
Error guessing	Experience-based guessing of likely faults	Empty input, zero, negative

Decision table example — a loan approval with two conditions:

Rule	Age ≥ 21?	Income ≥ 30k?	Action: Approve?
------	-----------	---------------	------------------

R1	T	T	Yes
R2	T	F	No
R3	F	T	No
R4	F	F	No

Two conditions → **2² = 4 rules**; each column is a test case. Decision tables ensure **every combination** is covered.

White-box testing (structural / glass-box)

Tests the **internal logic and structure of the code** — the tester sees the source. **Coverage criteria**, from weakest to strongest:

Coverage	Requires
Statement coverage	Every statement executed at least once
Branch (decision) coverage	Every branch (true <i>and</i> false of each decision) taken
Condition coverage	Every boolean sub-condition evaluated both ways
Path coverage	Every independent path through the code executed (strongest, often infeasible)

Branch coverage subsumes statement coverage (100% branch ⇒ 100% statement, not vice-versa) — a standard MCQ.

Cyclomatic complexity — the worked white-box metric

Cyclomatic complexity (McCabe) measures the number of **linearly independent paths** through a program — a bound on the minimum number of test cases needed for branch coverage, and a complexity/maintainability indicator. From the **control flow graph (CFG)** with **E** edges and **N** nodes:

$V(G) = E - N + 2P$ (P = number of connected components, usually 1)
 Equivalently **$V(G) = D + 1$** (D = number of **decision/predicate nodes**)
 Or **$V(G)$ = number of enclosed regions** in a planar CFG

All three give the **same** value — computing it two ways is a good exam check.

Worked example. Consider:

```

1  read x
2  if (x > 0)
3      print "positive"
   else
4      print "non-positive"
5  end if
6  print "done"
```

The CFG has decision node at line 2 with two branches merging before line 6.

- **Decisions $D = 1$** → $V(G) = D + 1 = 2$.

- **Nodes N : {1-2, 3, 4, 6} = 4; Edges E : 2→3, 2→4, 3→6, 4→6, plus 1→2 = 5.**

$V(G) = E - N + 2 = 5 - 4 + 2 = 3?$ — recount: model start→decision→two paths→merge→end as

N = 5 nodes (start, decision, then-branch, else-branch, end), E = 6 edges
 (start→decision, decision→then, decision→else, then→end, else→end, plus one entry). Using the reliable rule **$V(G) = D + 1 = 1 + 1 = 2$** : there are **2 independent paths** ($x > 0$ and $x \leq 0$), so **2 test cases** give full branch coverage. *(Always cross-check with $D + 1$ — it is the least error-prone form and the one to show.)*

Interpretation: $V(G) \leq 10$ is considered manageable; higher means the module is complex, error-prone and should be refactored. The value also equals the **minimum number of test cases** for branch coverage.

Other testing types

Type	Meaning
Alpha testing	By users/testers at the developer's site , before release
Beta testing	By real users in their own environment , pre-final release, feedback collected
Regression testing	Re-running existing tests after a change to ensure nothing previously working broke
Smoke / sanity	Quick check that the build is stable enough to test
Stress / load / performance	Behaviour under heavy or extreme load
Recovery / security / usability	Non-functional system tests

Likely exam questions

- **[5]** Differentiate between black-box and white-box testing.
- **[5]** Explain the levels of testing: unit, integration, system, acceptance.
- **[5]** Differentiate top-down and bottom-up integration. What are stubs and drivers?
- **[5]** Explain equivalence partitioning and boundary value analysis with an example.
- **[5]** Differentiate alpha and beta testing. What is regression testing?
- **[5]** Define cyclomatic complexity and state its three formulas.
- **[10]** Draw the control flow graph for a given program, compute its cyclomatic complexity by all three methods and state the number of independent paths.
- **[15]** Explain software testing in detail: the levels of testing, integration strategies, black-box techniques (EP, BVA, decision tables) and white-box coverage with cyclomatic complexity worked on an example.

MCQ traps

Trap	Correct answer
"Testing can prove the absence of bugs?"	No — only their presence.
"Error vs fault vs failure?"	Error (human) → fault (code) → failure (behaviour).
"Top-down integration uses?"	Stubs . Bottom-up uses drivers .
"BVA tests values at?"	The boundaries .

"Which testing needs source code?"	White-box ·
"Cyclomatic complexity = ?"	$E - N + 2P = D + 1 = \text{regions}$ ·
"Branch coverage subsumes?"	Statement coverage ·
"Beta testing is done?"	By real users at their own site ·
"Testing after a change to catch new breakage?"	Regression testing ·
"n conditions in a decision table give?"	2^n rules ·

11. Verification and Validation (V&V)

Concept

Two words that sound alike and are constantly confused — Boehm's one-liner nails them:

- **Verification** — "*Are we building the product right?*" — checks the software against its **specification** at each stage. **Static**, no execution: reviews, walkthroughs, inspections.
- **Validation** — "*Are we building the right product?*" — checks the finished software against the **user's actual needs**. **Dynamic**, requires execution: testing.

	Verification	Validation
Question	Building it right ?	Building the right thing?
Against	The specification/design	The user's requirements
Method	Static — reviews, inspections, walkthroughs	Dynamic — testing/execution
When	Throughout, at each phase	Mainly at the end
Finds	Deviations from spec	Wrong/missing requirements

Reviews/inspections (verification): **Fagan inspection** is a formal, role-based review (moderator, author, reader, tester) proven to catch defects far more cheaply than testing.

Walkthroughs are less formal.

Likely exam questions

- **[5]** Differentiate between verification and validation with examples.
- **[5]** What is a software inspection/review? How does it differ from testing?

MCQ traps

Trap	Correct answer
"Are we building the product right?"	Verification ·
"Are we building the right product?"	Validation ·

"Verification is static or dynamic?"	Static (no execution).
"Validation involves?"	Execution/testing .

12. Software metrics

Concept

Software metrics are **quantitative measures** of a product, process or project — "you cannot control what you cannot measure." Categories:

- **Product metrics** — size and quality of the software: **LOC/KLOC**, **function points**, **cyclomatic complexity**, **Halstead's software science** (operators/operands → volume, difficulty, effort), defect density (defects/KLOC).
- **Process metrics** — the development process: productivity (LOC or FP per person-month), defect removal efficiency, rework percentage.
- **Project metrics** — cost, effort, schedule, staffing.

LOC vs Function Points — the classic metrics comparison:

	LOC	Function Points
Measures	Physical size (lines)	Delivered functionality
Language-dependent?	Yes	No
Available	Only after coding	Early, from requirements
Weakness	Penalises concise code; varies by style	Subjective weighting

Halstead metrics (from n_1 , n_2 = distinct operators/operands; N_1 , N_2 = total counts):

Vocabulary $n = n_1 + n_2$; Length $N = N_1 + N_2$; **Volume** $V = N \times \log_2 n$; Difficulty $D =$

$(n_1/2) \times (N_2/n_2)$; Effort $E = D \times V$.

Likely exam questions

- **[5]** What are software metrics? Classify them.
- **[5]** Differentiate LOC and function points as size metrics.
- **[10]** Explain product, process and project metrics with examples, including Halstead's software science.

MCQ traps

Trap	Correct answer
"Which size metric is language-independent?"	Function points .
"Defects per KLOC is a?"	Product (quality) metric .
"Halstead volume $V = ?$ "	$N \times \log_2 n$.

"Cyclomatic complexity is a ____ metric?"

Product (complexity) metric.

13. Software Quality Assurance (SQA)

Concept

Software quality = conformance to **explicitly stated functional and performance requirements**, **explicitly documented development standards**, and **implicit characteristics** expected of professional software (Pressman). Note all three clauses — implicit expectations count too.

SQA is the **planned, systematic set of activities** that ensures quality is built into the process, not merely tested in at the end. It is **process-oriented** (prevention); **testing/QC** is **product-oriented** (detection).

SQA activities: defining standards and procedures; reviews, audits and inspections; **configuration management**; measurement (metrics); defect tracking; and reporting to management.

Quality control (QC) vs Quality assurance (QA): QC finds defects in the product (testing, inspection); QA improves the process to prevent defects. QA is the umbrella; QC is one activity within it.

Likely exam questions

- [5] Define software quality. What is SQA?
- [5] Differentiate quality assurance from quality control.
- [10] Explain the activities involved in software quality assurance.

MCQ traps

Trap	Correct answer
"QA is process- or product-oriented?"	Process (prevention). QC = product (detection).
"Software quality includes implicit requirements?"	Yes .
"Which is broader, QA or QC?"	QA .

14. Quality models — McCall, ISO 9126, CMM

McCall's quality model (1977)

Organises **11 quality factors** (external, user-visible) into **three perspectives**, each supported by internal **criteria**:

Perspective	Factors
Product operation	Correctness, Reliability, Efficiency, Integrity, Usability

Product revision	Maintainability, Flexibility, Testability
Product transition	Portability, Reusability, Interoperability

Mnemonic for the three perspectives: **operation** (how well it runs), **revision** (how easily it changes), **transition** (how well it moves to new environments).

ISO 9126 quality model

The international standard defining **six quality characteristics** (each with sub-characteristics):

Characteristic	Meaning
Functionality	Does it provide the required functions (suitability, accuracy, security)?
Reliability	Maturity, fault tolerance, recoverability
Usability	Understandability, learnability, operability
Efficiency	Time behaviour, resource use
Maintainability	Analysability, changeability, testability, stability
Portability	Adaptability, installability, replaceability

(Superseded by **ISO/IEC 25010**, which adds **security** and **compatibility** as top-level characteristics — mention it.)

CMM — Capability Maturity Model (SEI) — levels 1–5

The **CMM** rates an **organisation's software-process maturity** on **five levels**; each level builds on the one below through **Key Process Areas (KPAs)**. This is a guaranteed exam item — learn the five levels and their one-word essence.

Level	Name	Essence	Character
1	Initial	Ad hoc, chaotic	Success depends on individual heroics; unpredictable
2	Repeatable (Managed)	Basic project management	Processes for cost/schedule/tracking; can repeat earlier successes
3	Defined	Standardised, documented organisation-wide process	A tailored standard process for all projects
4	Managed (Quantitatively Managed)	Measured and controlled	Process and product quantitatively understood via metrics
5	Optimising	Continuous improvement	Feedback and innovation continuously improve the process

Mnemonic: **I**nitial, **R**epeatable, **D**efined, **M**anaged, **O**ptimising →

"I Really Do Manage Operations." Note: **Level 1 has no KPAs**; higher levels

each add KPAs (e.g. Level 2: requirements management, project planning, configuration management; Level 5: defect prevention, process/technology change management). CMM has evolved into

CMMI.

Likely exam questions

- [5] State the three product perspectives of McCall's quality model with their factors.
- [5] List the six quality characteristics of ISO 9126.
- [5] List and briefly explain the five levels of the CMM.
- [10] Explain the CMM. Describe its five maturity levels and their key process areas.
- [10] Compare McCall's model, ISO 9126 and the CMM as approaches to software quality.

MCQ traps

Trap	Correct answer
"How many levels in the CMM?"	Five (Initial → Optimising).
"CMM Level 3 is?"	Defined . Level 5 = Optimising .
"Which CMM level has no KPAs?"	Level 1 (Initial) .
"ISO 9126 defines how many characteristics?"	Six .
"McCall's model has how many factors?"	11 , in 3 perspectives.
"Maintainability belongs to which McCall perspective?"	Product revision .

15. Software reliability

Concept

Software reliability is the **probability of failure-free operation for a specified time in a specified environment**. Unlike hardware, software failures come from **design/coding defects**, not wear — so reliability *grows* as defects are found and fixed (**reliability growth models**, e.g. Jelinski-Moranda, Musa).

The key measures (and the relationship among them):

Metric	Meaning
MTTF — Mean Time To Failure	Average time the system runs before a failure (non-repairable)
MTTR — Mean Time To Repair	Average time to fix and restore after a failure
MTBF — Mean Time Between Failures	MTBF = MTTF + MTTR
Availability	MTTF / (MTTF + MTTR) = MTTF / MTBF , often as a %
Failure rate (λ)	Failures per unit time; Reliability $R(t) = e^{(-\lambda t)}$

Worked example. If MTTF = 970 hours and MTTR = 30 hours, then MTBF = 1000 hours and **availability = 970/1000 = 97%**.

Clean-room approach (named in the Forensic syllabus): a methodology that emphasises **defect prevention rather than removal** — rigorous specification, **formal correctness verification** (mathematical proofs / correctness arguments) instead of unit debugging, and **statistical testing** based on the expected usage profile to certify reliability. The aim is to produce software with very low defect density on first release.

Likely exam questions

- [5] Define software reliability. Differentiate MTTF, MTTR and MTBF.
- [5] How does software reliability differ from hardware reliability?
- [5] What is the clean-room approach to software development?
- [10] Explain software reliability, its metrics and reliability growth, and compute availability from given MTTF/MTTR.

MCQ traps

Trap	Correct answer
"MTBF = ?"	MTTF + MTTR .
"Availability = ?"	MTTF / (MTTF + MTTR) .
"Reliability formula R(t)?"	$e^{(-\lambda t)}$.
"Clean-room emphasises?"	Defect prevention + formal verification + statistical testing.
"Software fails due to?"	Design/coding defects , not wear.

16. Software maintenance

Concept

Maintenance is modifying software **after delivery**. It is the **longest and costliest** phase (~60–70% of lifetime cost). Four types — a guaranteed MCQ and 5-marker:

Type	Trigger	Purpose	Share
Corrective	A fault/bug reported	Fix errors	~20%
Adaptive	A changed environment (new OS, hardware, law)	Keep it working in the new environment	~25%
Perfective	User requests new/better features	Enhance functionality/performance	~50% (largest)
Preventive	Latent problems / ageing code	Prevent future faults; improve maintainability (re-documentation, restructuring)	~5%

Perfective maintenance is the largest slice — most maintenance is *enhancement*, not bug-fixing. That surprises students and is a favourite MCQ.

Lehman's laws of software evolution (worth a mention): software must **continually change** or become less useful (continuing change); and as it changes its **complexity increases** unless work is done to reduce it (increasing complexity).

Likely exam questions

- **[5]** What is software maintenance? Explain its four types.
- **[5]** Which type of maintenance consumes the most effort and why?
- **[10]** Explain the types of software maintenance and the factors that make maintenance difficult.

MCQ traps

Trap	Correct answer
"Fixing reported bugs is?"	Corrective maintenance.
"Adapting to a new OS is?"	Adaptive maintenance.
"Adding new features is?"	Perfective maintenance.
"Which type consumes the most effort?"	Perfective (~50%).
"Improving future maintainability is?"	Preventive maintenance.

17. Reengineering and reverse engineering

Concept

Both deal with **existing (legacy) systems** and are named explicitly in the Forensic syllabus.

- **Reverse engineering** — analysing an existing system to **recover its design/specification**, working **backward** from code to higher-level abstractions (design, then requirements). It **only extracts information; it does not change the system**. Used to understand undocumented legacy code. (In the Forensic context, reverse engineering of *binaries/malware* is analysing compiled code to recover its behaviour.)
- **Forward engineering** — the normal direction: requirements → design → code.
- **(Software) Reengineering** — **examining and altering** an existing system to **reconstitute it in a new, improved form** while **preserving its functionality**. It is essentially **reverse engineering followed by forward engineering**: understand the old system, then rebuild it with better structure, technology or documentation. Activities: inventory analysis, document restructuring, reverse engineering, **code restructuring**, **data restructuring**, forward engineering.
- **Restructuring** — changing code/data structure **without changing functionality** (e.g. converting spaghetti code to structured code); a component of reengineering.

The relationship to state: *reengineering* = *reverse engineering* + (*restructuring*) + *forward engineering*. Reverse engineering **understands**; reengineering **rebuilds**.

Likely exam questions

- **[5]** Differentiate reverse engineering and reengineering.
- **[5]** What is software restructuring? How does it relate to reengineering?
- **[10]** Explain software reengineering, its process and its relationship with reverse and forward engineering.

MCQ traps

Trap	Correct answer
"Recovering design from code is?"	Reverse engineering.
"Rebuilding a legacy system in improved form is?"	Reengineering.
"Reengineering = ?"	Reverse engineering + forward engineering.
"Changing structure without changing behaviour is?"	Restructuring.
"Does reverse engineering modify the system?"	No — it only extracts information.

18. Last-week revision sheet

Twenty facts most likely to be one-mark MCQs:

1. Software **deteriorates, does not wear out**; software = **programs + data + documentation**.
2. **Software crisis** → over budget/schedule, poor quality; term from the **1968 NATO** meeting.
3. SDLC phases: requirements → design → coding → testing → deployment → **maintenance (longest/costliest, ~60–70%)**.
4. **Waterfall** = fixed requirements, no early product; **Prototype** = unclear requirements; **Spiral (Boehm)** = **risk-driven**; **V-model** = each phase paired with a test phase; **Agile** = volatile requirements, continuous delivery.
5. **SRS** must be correct, complete, **unambiguous**, consistent, **verifiable**, traceable (**IEEE 830**).
6. **Feasibility** = **TELOS** (Technical, Economic, Legal, Operational, Schedule).
7. **Brooks's Law**: adding people to a late project makes it later.
8. Basic **COCOMO** $E = a(KLOC)^b$, $D = 2.5 \cdot E^d$; organic **a=2.4, b=1.05**; embedded highest **b=1.20**. Intermediate adds **EAF** (15 cost drivers). **Boehm, 1981**.
9. **Function points** are **language-independent**, computed early; five types **EI, EO, EQ, ILF, EIF**; $FP = UFP \times VAF$, $VAF \in [0.65, 1.35]$.
10. **Critical path** = **longest path** = **minimum project duration**; critical activities have **zero slack**. $PERT\ t_e = (o + 4m + p)/6$.
11. **Gantt** shows timeline (not dependencies); **PERT/CPM** shows dependencies; **CPM** = **deterministic**, **PERT** = **probabilistic**.
12. Design goal: **low coupling, high cohesion**.
13. Coupling best→worst: **Data, Stamp, Control, External, Common, Content**.
14. Cohesion worst→best: **Coincidental, Logical, Temporal, Procedural, Communicational**,

Sequential, Functional.

15. **Unit** uses stubs/drivers; **top-down = stubs, bottom-up = drivers; acceptance** by the customer; **alpha** at developer site, **beta** at user site; **regression** after changes.
16. **Black-box** = spec-based (EP, BVA, decision tables); **white-box** = code-based; **branch coverage subsumes statement coverage**.
17. **Cyclomatic complexity** $V(G) = E - N + 2 = D + 1 = \text{regions}$ = independent paths = min test cases for branch coverage.
18. **Verification** = "building it right?" (static); **Validation** = "right product?" (dynamic).
19. **CMM levels: 1 Initial, 2 Repeatable, 3 Defined, 4 Managed, 5 Optimising** (Level 1 has no KPAs). **ISO 9126** = 6 characteristics; **McCall** = 11 factors in 3 perspectives.
20. Maintenance types: corrective (bugs), adaptive (environment), **perfective (features — largest ~50%)**, preventive; **reengineering = reverse + forward engineering**; **MTBF = MTTF + MTTR**, availability = MTTF/MTBF.

Descriptive-paper strategy. Software engineering reliably supplies Section-A questions in all three papers and at least one 15-mark question you can fully answer. Prepare these six as guaranteed long answers, each with its diagram or worked steps:

- (a) **process models compared** (Waterfall, Prototype, Spiral, V-model, Agile) with diagrams — draw each model *first*;
- (b) **coupling and cohesion** fully ranked with examples — a pure-recall high scorer;
- (c) **COCOMO + function points** with one worked numerical each — unambiguous to mark;
- (d) **PERT/CPM** — draw the network, find the critical path and slack;
- (e) **testing** — levels, integration strategies, black/white-box, and **cyclomatic complexity worked on a CFG**;
- (f) **CMM's five levels** and **ISO 9126 / McCall** quality models.

Always draw the diagram or lay out the worked calculation *first*, label it, then write the prose — if you run out of time, a labelled diagram or a correct numerical still scores.

© Degree — MCQ bank

Source: ComputerScience_Degree/CSD_05_MCQ_and_QuestionBank.md

CSD_05 — MCQ Bank & Descriptive Question Bank (Full CS Degree Syllabus)

Covers the **MCQ paper** ($100 \times 2 = 200$ marks, $-1/3$ negative marking) and the **descriptive Paper-I / Paper-II** (each 100 marks, Sec A 8-of-10 $\times$ 5, Sec B 3-of-5 $\times$ 10, Sec C 2-of-4 $\times$ 15), across the whole CS (Degree) syllabus in **OFFICIAL_SYLLABUS.md** §3 — Basics & Programming, Data Structures & Algorithms, Logic Design, Computer Organization & Architecture, System Software & Compilers, Mobile Applications (Paper-I); Operating Systems, Software Engineering, DBMS, Computer Networks, Web Technologies, AI, Machine Learning (Paper-II).

This file is a **drilling tool**, not a teaching one — it assumes you have read the topic files (**CSD_00–CSD_04** and the relevant **CORE_*** files) and are now testing recall and exam-writing

speed. Difficulty is calibrated to the solved 2025 CS-Degree papers in [_raw/existing_txt/](#).

How to use this file

- [] Attempt each MCQ block of 10 **closed-book**, then check against the key.
- [] For every five-mark skeleton, write the actual bullet points from memory in under 4 minutes; for ten-mark in under 8; for fifteen-mark in under 12 — that is your real per-question time budget (180 minutes ÷ 13 questions answered ≈ 14 min each, with slack for reading/choosing).
- [] Re-attempt only the MCQs you got wrong or guessed, weekly, until the block is clean.

The negative-marking rule — read this before you touch the MCQ paper

MCQ is 100 questions × 2 marks = 200 marks; each **wrong** answer costs **−1/3 mark actual**, i.e. $-1/3 \times 2 = \mathbf{-0.667 \text{ marks}}$ (some official notations state the penalty as $-1/3$ of the question's own mark value; either way, treat it as **losing a third of what a correct answer would have earned**). An **unattempted** question costs exactly 0.

The maths that decides whether to guess:

Strategy	Expected value per question (4 options, uniform prior)
Leave blank	0
Guess blind among all 4 options	$(1/4)(+2) + (3/4)(-2/3) = 0.5 - 0.5 = \mathbf{0}$ (exact break-even)
Guess among 3 options (1 eliminated)	$(1/3)(+2) + (2/3)(-2/3) = 0.667 - 0.444 = \mathbf{+0.222}$ (positive)
Guess among 2 options (2 eliminated)	$(1/2)(+2) + (1/2)(-2/3) = 1 - 0.333 = \mathbf{+0.667}$ (strongly positive)
Answer with genuine confidence	Expected value → +2 as confidence → certainty

The rule in one line: a pure blind 4-way guess is exactly break-even — it costs you nothing in expectation, but it does add variance for no expected gain, so there is no reason to blind-guess. The moment you can eliminate even one wrong option, the guess turns strictly profitable — always guess once you've eliminated at least one distractor. Never leave a question blank just because you are unsure between three options; leave it blank only when you truly have zero information to eliminate any option (rare — most "trap" options in this paper are eliminable by basic reasoning: wrong units, wrong data type, wrong direction of a comparison).

PART 1 — MCQ BANK (120 questions, grouped in blocks of 10)

Block 1 — Programming in C/C++ (Q1–10)

1. What is the output of `printf("%d", 7 / 2);` in C?
A) 3.5 B) 3 C) 4 D) Compile error
2. Which header is required for `malloc()` in C?
A) `<stdio.h>` B) `<stdlib.h>` C) `<memory.h>` D) `<conio.h>`
3. What does `strcmp("abc", "abc")` return?
A) 1 B) -1 C) 0 D) The address of the string
4. What is the scope of a `static` local variable in C?
A) Global to the whole program B) Local to the function, but retains value between calls C) Local to the block only D) File scope only
5. In C++, which mechanism allows a derived class to provide its own version of a base class function?
A) Overloading B) Overriding (via virtual functions) C) Templates D) Friend functions
6. What is the size (in bytes, typical 32/64-bit system) of a `char` in C?
A) 1 B) 2 C) 4 D) 8
7. Which of these is NOT a valid C storage class?
A) `auto` B) `register` C) `extern` D) `dynamic`
8. What does the `volatile` keyword signify in C?
A) The variable is thread-safe B) The compiler must not optimize away reads/writes to this variable C) The variable is stored in ROM D) The variable cannot change
9. In C++, what is function overloading?
A) Same function name with different implementations at runtime B) Same function name with different parameter lists resolved at compile time C) A function that calls itself D) A virtual function
10. What is a dangling pointer?
A) A pointer that has never been initialised B) A pointer that points to memory that has been freed/deallocated C) A NULL pointer D) A pointer to a function

Block 2 — Data Structures (Q11–20)

11. Which data structure uses LIFO (Last In First Out) order?
A) Queue B) Stack C) Linked list D) Tree
12. The worst-case time complexity of searching in a balanced binary search tree is:
A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n \log n)$
13. Which sorting algorithm is stable AND has $\Theta(n \log n)$ worst-case complexity?
A) Quick sort B) Heap sort C) Merge sort D) Selection sort
14. In a hash table, when two keys map to the same slot, this is called:
A) Overflow B) Collision C) Rehashing D) Aliasing
15. A queue implemented using two stacks needs how many stacks reversed to dequeue?
A) Not possible B) One C) Two D) It depends on data
16. What is the time complexity of inserting at the head of a singly linked list?
A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
17. Which traversal of a binary search tree yields elements in sorted order?
A) Preorder B) Inorder C) Postorder D) Level order
18. A complete binary tree with n nodes has height:
A) $O(n)$ B) $O(\log n)$ C) $O(n^2)$ D) $O(\sqrt{n})$
19. What is the auxiliary space complexity of standard (non-in-place) merge sort?

- A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
20. In a min-heap, where is the smallest element always located?
A) Last leaf B) Root C) Rightmost node D) Undetermined

Block 3 — Algorithm Analysis (Q21–30)

21. What is the time complexity of linear search in the worst case?
A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
22. Which notation gives an asymptotically tight bound?
A) Big-O B) Big-Omega C) Big-Theta D) little-o
23. The recurrence $T(n) = 2T(n/2) + n$ solves to:
A) $\Theta(n)$ B) $\Theta(n \log n)$ C) $\Theta(n^2)$ D) $\Theta(\log n)$
24. Quick sort's worst-case time complexity occurs when the input array is:
A) Random B) Already sorted (with first/last-element pivot) C) All equal elements only D) Reverse of random
25. Which paradigm builds a solution by always taking the locally optimal choice?
A) Divide and conquer B) Dynamic programming C) Greedy D) Backtracking
26. Dynamic programming is applicable when a problem exhibits:
A) Greedy-choice property only B) Optimal substructure and overlapping subproblems C) NP-completeness D) No recursive structure
27. Which of these problems is solved optimally by a greedy algorithm?
A) 0/1 Knapsack B) Fractional Knapsack C) Longest Common Subsequence D) Travelling Salesman
28. A problem is in NP if:
A) It can be solved in polynomial time B) A proposed solution can be verified in polynomial time C) It cannot be solved at all D) It requires exponential space
29. Which traversal algorithm uses a queue internally?
A) DFS B) BFS C) Inorder traversal D) Postorder traversal
30. Strassen's matrix multiplication algorithm reduces the number of multiplications per recursive step from 8 to:
A) 6 B) 7 C) 5 D) 4

Block 4 — Digital Logic & Number Systems (Q31–40)

31. The 2's complement of the 8-bit binary number 00001010 is:
A) 11110101 B) 11110110 C) 11110100 D) 00001010
32. Which gate produces an output of 1 only when both inputs differ?
A) AND B) OR C) XOR D) NAND
33. How many flip-flops are needed to build a MOD-16 counter?
A) 2 B) 4 C) 8 D) 16
34. NAND gate is called a "universal gate" because:
A) It is the fastest gate B) Any Boolean function can be implemented using only NAND gates C) It has the lowest power consumption D) It was invented first
35. The hexadecimal equivalent of decimal 255 is:
A) FF B) FE C) 100 D) F0

36. A K-map is used primarily for:
A) Sequential circuit design B) Minimising Boolean expressions C) Memory addressing D) Clock synchronisation
37. A JK flip-flop with J=1, K=1 on a clock edge will:
A) Set to 1 B) Reset to 0 C) Toggle its previous state D) Remain unchanged (no-change/hold)
38. A multiplexer with n select lines can choose among how many inputs?
A) n B) n^2 C) 2n D) 2^n
39. In 1's complement representation, what causes the "end-around carry" adjustment?
A) Overflow of the sign bit only B) A carry-out from the MSB, added back to the LSB C) Underflow D) Nothing — 1's complement has no such adjustment
40. A half adder produces which two outputs?
A) Sum and Carry B) Sum and Borrow C) Carry and Overflow D) Product and Remainder

Block 5 — Computer Organization & Architecture (Q41–50)

41. Which register holds the address of the next instruction to be fetched?
A) Accumulator B) Program Counter (PC) C) Instruction Register (IR) D) Memory Address Register (MAR)
42. Cache memory is used primarily to:
A) Increase disk storage B) Bridge the speed gap between CPU and main memory C) Store the operating system permanently D) Manage virtual memory pages
43. DMA (Direct Memory Access) allows:
A) The CPU to access memory faster B) Peripheral devices to transfer data to/from memory without CPU intervention for each word C) Two CPUs to share memory D) Memory to access the disk directly
44. Which addressing mode uses the operand's address stored in a register?
A) Immediate B) Direct C) Register indirect D) Indexed
45. The technique of overlapping fetch, decode and execute of successive instructions is called:
A) Multiprogramming B) Pipelining C) Multitasking D) Interleaving
46. Which memory is the fastest but most expensive per bit?
A) Main memory (RAM) B) Cache memory (built from registers/SRAM) C) Magnetic disk D) Optical disk
47. Which of these is a cause of a structural hazard in a pipeline?
A) Data dependency between instructions B) Two instructions needing the same hardware resource simultaneously C) A branch instruction changing control flow D) An interrupt
48. What does "memory interleaving" primarily improve?
A) Memory capacity B) Memory access bandwidth by allowing overlapped access across banks C) Memory security D) Cache coherence
49. In the immediate addressing mode, the operand is:
A) In a register B) In memory, address given directly C) Part of the instruction itself D) In memory, address computed from a base register
50. Which unit of the CPU performs arithmetic and logical operations?
A) Control Unit B) ALU C) Cache D) MMU

Block 6 — System Software & Compilers (Q51–60)

51. Which phase of a compiler builds the symbol table entries and checks type compatibility?

- A) Lexical analysis B) Syntax analysis C) Semantic analysis D) Code generation
52. A "token" in compiler terminology is produced by:
A) The parser B) The lexical analyser (scanner) C) The code generator D) The linker
53. Which parsing technique processes input from left to right, building the rightmost derivation in reverse?
A) LL parsing B) LR parsing C) Recursive descent D) Operator precedence
54. A macro, as used by an assembler, is:
A) A hardware instruction B) A named template of code expanded inline at each call, at assembly/compile time C) A runtime function call D) A type of loader
55. The main job of a linker is to:
A) Convert source code to assembly B) Combine object modules and resolve external references into an executable C) Load a program into memory for execution D) Detect syntax errors
56. Which loader scheme relocates a program's addresses at load time, allowing it to be loaded anywhere in memory?
A) Absolute loader B) Relocating loader C) Direct linking loader D) Bootstrap loader
57. Dead code elimination and constant folding are examples of:
A) Lexical analysis techniques B) Code optimisation techniques C) Syntax error recovery D) Symbol table operations
58. Which grammar class corresponds to context-free languages, relevant to most programming language syntax?
A) Type 0 (unrestricted) B) Type 1 (context-sensitive) C) Type 2 (context-free) D) Type 3 (regular)
59. An "editor" in the context of system software refers to:
A) A program that translates source code B) A tool for creating/modifying source text files C) A device driver D) A scheduling algorithm
60. A three-address code intermediate representation typically has the form:
A) $x = y \text{ op } z$ B) $x = y$ C) A parse tree only D) Raw machine code

Block 7 — Operating Systems (Q61–70)

61. Which CPU scheduling algorithm can cause starvation of long processes?
A) FCFS B) Round Robin C) Shortest Job First (SJF) D) None of these
62. A deadlock can occur only if all of the following conditions hold simultaneously EXCEPT:
A) Mutual exclusion B) Hold and wait C) Preemption D) Circular wait
(Note: correct deadlock conditions are Mutual exclusion, Hold and Wait, No Preemption, Circular Wait — "Preemption" as stated, i.e. allowing preemption, is the one that must NOT hold.)
63. Which memory management technique suffers from external fragmentation?
A) Paging B) Segmentation with variable-sized partitions C) Fixed-size paging only D) None
64. The "belady's anomaly" refers to:
A) A CPU scheduling paradox B) Increasing page faults despite increasing the number of frames, under FIFO page replacement C) A deadlock detection failure D) A file system corruption bug
65. Which page replacement algorithm is provably optimal (minimises page faults) but not implementable in practice?
A) FIFO B) LRU C) Optimal (OPT/Belady's algorithm) D) Second-chance
66. A critical section problem solution must satisfy:
A) Mutual exclusion, Progress, Bounded waiting B) Only mutual exclusion C) Only fairness D) Deadlock freedom alone, without mutual exclusion

85. Which type of SQL join returns all rows from the left table and matched rows from the right, with NULLs where there is no match?
A) INNER JOIN B) LEFT OUTER JOIN C) RIGHT OUTER JOIN D) CROSS JOIN
86. A transaction's "isolation" property in ACID ensures:
A) All operations complete or none do B) The database remains in a consistent state C) Concurrent transactions do not interfere with each other's intermediate states D) Committed changes survive a crash

Block 10 — Computer Networks (Q87–94)

87. Which OSI layer is responsible for routing packets between different networks?
A) Data Link layer B) Network layer C) Transport layer D) Session layer
88. TCP differs from UDP primarily in that TCP is:
A) Faster but unreliable B) Connection-oriented and reliable (with acknowledgment/retransmission) C) Used only for streaming D) Stateless
89. A subnet mask of 255.255.255.0 corresponds to a CIDR prefix of:
A) /16 B) /24 C) /8 D) /32
90. Which device operates at the Data Link layer, forwarding frames based on MAC addresses?
A) Hub B) Switch C) Router D) Repeater
91. DNS primarily performs:
A) Encryption of network traffic B) Translation of domain names to IP addresses C) Routing table calculation D) Error correction in transmission
92. Which of these is a Class C private IP address range?
A) 10.0.0.0 – 10.255.255.255 B) 172.16.0.0 – 172.31.255.255 C) 192.168.0.0 – 192.168.255.255 D) 127.0.0.0 – 127.255.255.255
93. The three-way handshake used to establish a TCP connection is:
A) SYN, SYN, ACK B) SYN, SYN-ACK, ACK C) ACK, SYN, FIN D) SYN, ACK, FIN
94. Which topology has every node connected directly to every other node?
A) Star B) Bus C) Mesh (full mesh) D) Ring

Block 11 — Web Technologies (HTML/CSS/PHP) (Q95–104)

95. In an HTML form, which attribute determines where the form data is sent?
A) `method` B) `action` C) `target` D) `name`
96. Which PHP superglobal array holds data submitted via the POST method?
A) `$_GET` B) `$_POST` C) `$_REQUEST` only D) `$_SESSION`
97. Which HTML tag is used to define an unordered (bulleted) list?
A) `<ol>` B) `<ul>` C) `<dl>` D) `<list>`
98. In PHP, which comparison operator checks both value and type equality?
A) `==` B) `===` C) `=` D) `<=>`
99. Where does data sent via the GET method appear?
A) In the HTTP request body only B) Appended to the URL as a query string C) In a cookie automatically D) Nowhere visible
100. In PHP, session data is stored:
A) In the client's browser as plain text B) On the server, with only a session ID held client-side C) In the

- HTML source code D) In the URL always
101. What is the correct file extension for a server to invoke the PHP interpreter?
A) `.html` B) `.php` C) `.htm` D) `.js`
102. Which CSS property controls the space between an element's border and its content?
A) `margin` B) `padding` C) `border-spacing` D) `outline`
103. The `alt` attribute of an `<img>` tag is used for:
A) Setting image alignment B) Alternate text shown if the image fails to load / read by screen readers C) Setting image width D) Compressing the image
104. Which HTTP status code indicates the requested resource was not found?
A) 200 B) 301 C) 404 D) 500

Block 12 — Mobile Applications (Q105–112)

105. Which of the four Android components has no user interface and runs in the background?
A) Activity B) Service C) Content Provider D) Fragment (not one of the four)
106. The Android callback invoked when an activity becomes fully invisible is:
A) `onPause()` B) `onStop()` C) `onDestroy()` D) `onCreate()`
107. The minimum recommended touch target size on Android is approximately:
A) 24×24 dp B) 48×48 dp C) 100×100 dp D) 16×16 dp
108. Which file in an Android project declares all its components and permissions?
A) `build.gradle` B) `AndroidManifest.xml` C) `strings.xml` D) `MainActivity.java`
109. Flutter compiles application code to:
A) Interpreted JavaScript B) Native ARM machine code via its own rendering engine C) A hybrid WebView wrapper only D) Bytecode requiring a JVM
110. An "implicit intent" in Android:
A) Names the exact target component by class B) Declares a general action and lets the OS resolve a suitable handling component C) Is used only for starting services D) Cannot carry any data
111. What is the file extension of a signed, installable Android application package?
A) `.exe` B) `.apk` C) `.jar` D) `.dmg`
112. Local SQLite storage on Android is most commonly accessed today through which abstraction library?
A) Realm only B) Room C) Core Data D) Retrofit

Block 13 — AI (Q113–116)

113. In propositional/first-order logic used for AI knowledge representation, "Modus Ponens" allows inferring:
A) $\neg P$ from $P \rightarrow Q$ B) Q from P and $P \rightarrow Q$ C) P from Q D) Nothing valid
114. Inductive learning generalises a hypothesis primarily from:
A) A single labelled example B) A set of specific examples to a general rule C) An expert-provided rule base only D) Random search alone
115. In PROLOG, a program is fundamentally composed of:
A) Sequential imperative statements B) Facts and rules, queried by unification and backtracking C) Only mathematical functions D) Object classes and methods
116. Which of these best describes "explanation-based learning" (EBL)?
A) Learning purely from statistical correlation in big data B) Using a single example plus prior domain

knowledge to generalise a learned rule via deductive analysis C) Unsupervised clustering D) Random trial and error

Block 14 — Machine Learning (Q117–120)

117. K-Nearest Neighbours is an example of:
A) An unsupervised clustering algorithm B) A supervised, instance-based (lazy) learning algorithm C) A reinforcement learning algorithm D) A dimensionality reduction technique
118. K-means clustering minimises which objective?
A) Cross-entropy loss B) Within-cluster sum of squared distances to the centroid C) Number of clusters D) Margin between classes
119. Bagging (Bootstrap Aggregating) primarily reduces:
A) Bias B) Variance, by averaging predictions of models trained on bootstrapped samples C) Training time only D) The need for any labelled data
120. Boosting (e.g. AdaBoost) builds an ensemble by:
A) Training all models independently and averaging B) Sequentially training models, each focusing more on the previous model's errors C) Randomly discarding weak learners D) Clustering the training data first

Answer key and explanations

Q	Ans	Why	Common wrong-option reason
1	B	Integer division in C truncates toward zero: $7/2 = 3$	A assumes float division
2	B	<code>malloc</code> , <code>calloc</code> , <code>free</code> declared in <code>stdlib.h</code>	<code>stdio.h</code> is I/O only
3	C	Identical strings $\rightarrow$ <code>strcmp</code> returns 0	1/-1 are for "greater/less than" cases
4	B	<code>static</code> locals persist across calls but stay function-scoped	Confused with global scope
5	B	Runtime polymorphism via virtual functions is overriding	Overloading is compile-time, same name/diff signature
6	A	<code>char</code> is 1 byte (guaranteed by the C standard)	—
7	D	<code>auto</code> , <code>register</code> , <code>static</code> , <code>extern</code> are the four; "dynamic" is not a C keyword	—
8	B	<code>volatile</code> prevents compiler optimisation of reads/writes (value may change outside program flow)	Thread-safety is not what <code>volatile</code> guarantees in C
9	B	Overloading resolved at compile time by signature	A describes overriding (runtime)
10	B	Freed memory still pointed to = dangling pointer	Uninitialised pointer is a "wild" pointer, a related but distinct term
11	B	Stack = LIFO by definition	Queue is FIFO

12	B	Balanced BST height is $O(\log n)$, so search is $O(\log n)$	Unbalanced BST could be $O(n)$, but "balanced" is specified
13	C	Merge sort: stable and $\Theta(n \log n)$ worst case	Quick sort worst is $\Theta(n^2)$; heap sort is $\Theta(n \log n)$ but NOT stable
14	B	Two keys $\rightarrow$ same slot = collision	Overflow is a related but distinct term (bucket full)
15	C	Classic "queue using two stacks" needs both stacks	One-stack solutions do not give $O(1)$ amortised dequeue+enqueue split cleanly
16	A	Insert at head is a fixed number of pointer updates: $O(1)$	Insert at tail (no tail pointer) would be $O(n)$
17	B	Inorder (Left-Root-Right) on a BST yields sorted order	Preorder/postorder do not sort
18	B	Complete binary tree height is $O(\log n)$ by definition	—
19	C	Standard merge sort needs an $O(n)$ auxiliary buffer for merging	In-place variants exist but are non-standard/slower
20	B	Min-heap property: root is always the minimum	Heaps are not sorted arrays; only the root guarantee holds
21	C	Linear search worst case scans all n elements: $O(n)$	—
22	C	Θ gives matching upper AND lower bound = tight	O is upper only, Ω is lower only
23	B	Master theorem case 2: $a=2, b=2, f(n)=n, e=1 \rightarrow \Theta(n \log n)$	—
24	B	Already-sorted input with a bad (first/last) pivot choice gives $O(n^2)$ splits every time	Random input gives the average case, not worst
25	C	Greedy = locally optimal choice each step, no backtracking	DP evaluates all options via subproblems
26	B	DP requires optimal substructure AND overlapping subproblems	Greedy-choice property belongs to greedy, not DP
27	B	Fractional knapsack: greedy on value/weight ratio is provably optimal	0/1 knapsack fails greedy, needs DP
28	B	NP = solutions verifiable in polynomial time	Solvable in poly time defines P, not NP
29	B	BFS explicitly uses a queue (level-by-level)	DFS uses a stack/recursion
30	B	Strassen reduces 8 multiplications to 7 (at the cost of more additions)	—

31	A	00001010 → invert = 11110101, +1 = 11110110... (recompute: invert 00001010 → 11110101, +1 = 11110110) → B is correct, 11110110	(see corrected note below the table)
32	C	XOR outputs 1 when inputs differ	AND/OR do not have this "difference detector" property
33	B	MOD-16 = 2^4 , needs 4 flip-flops	2 flip-flops only gives MOD-4
34	B	NAND (and NOR) alone can implement any Boolean function — universal gate	Not about speed or history
35	A	255 decimal = FF hex ($15 \times 16 + 15$)	—
36	B	Karnaugh maps minimise Boolean/SOP-POS expressions	Not used for sequential design directly
37	C	JK flip-flop with J=K=1 toggles on clock edge	J=K=0 is hold; J=1, K=0 is set; J=0, K=1 is reset
38	D	n select lines choose among 2^n inputs	Common MCQ trap swapping n and 2^n
39	B	1's complement addition: carry-out from MSB wraps around and adds to LSB	2's complement addition simply discards the final carry
40	A	Half adder: Sum and Carry outputs only (no carry-in)	Full adder additionally takes a carry-in
41	B	Program Counter holds address of the NEXT instruction	IR holds the CURRENT instruction being decoded
42	B	Cache bridges CPU-memory speed gap	Not disk-related
43	B	DMA transfers data directly without per-word CPU involvement	CPU is freed up, not sped up directly
44	C	Register indirect: register holds the address, not the value	Direct addressing stores the address in the instruction itself
45	B	Overlapping instruction stages = pipelining	Multiprogramming/multitasking are OS-level concepts, different layer
46	B	Cache (SRAM) is faster and costlier per bit than RAM	RAM is cheaper/slower than cache
47	B	Structural hazard = resource contention (e.g. one memory port needed by two stages)	Data hazard is about data dependency
48	B	Interleaving spreads addresses across banks for overlapped, higher-bandwidth access	Not a capacity or security feature
49	C	Immediate addressing: the operand's actual value is embedded in the instruction	Direct addressing gives an address, not the value itself

50	B	ALU = Arithmetic Logic Unit performs computation	Control Unit directs, does not compute
51	C	Semantic analysis does type checking and symbol table use/verification	Lexical analysis only tokenises
52	B	The lexical analyser (scanner) groups characters into tokens	Parser consumes tokens, does not produce them
53	B	LR parsing builds a rightmost derivation in reverse, scanning left to right	LL builds leftmost derivation
54	B	A macro is expanded inline at assembly/compile time by the preprocessor/assembler	Not a runtime call
55	B	Linker combines object files, resolves external symbol references	Loader places the executable in memory; different stage
56	B	Relocating loader adjusts addresses so the program can load anywhere	Absolute loader requires a fixed load address
57	B	Both are classic optimisation-phase techniques	Not lexical or symbol-table operations
58	C	Most PL syntax is context-free (Type 2 in the Chomsky hierarchy)	Regular (Type 3) cannot express nested structures like balanced parentheses
59	B	An editor creates/modifies source text	Not a translator or scheduler
60	A	Three-address code form: one operator, at most two operands, one result	$x=y$ is a simpler special case, not the general form
61	C	SJF can starve long jobs if short jobs keep arriving	FCFS/RR do not starve by job length
62	C	The four necessary conditions are Mutual Exclusion, Hold-and-Wait, No Preemption, Circular Wait — so "Preemption" (allowing it) breaks the deadlock condition set; it is the one that must NOT hold for deadlock	This question tests whether you know "No Preemption" (not "Preemption") is the actual condition
63	B	Variable-sized segmentation/partitions can leave unusable external gaps	Fixed-size paging causes internal, not external, fragmentation
64	B	Belady's anomaly: FIFO can have MORE page faults with MORE frames — counter-intuitive	Not related to scheduling or deadlock
65	C	Optimal (Belady's) algorithm needs future knowledge — theoretical benchmark only	LRU/FIFO are practical approximations
66	A	Mutual exclusion + Progress + Bounded waiting are the three required properties	Fairness alone is insufficient; deadlock-freedom alone is insufficient

67	B	User-level threads: kernel sees only the process, so one blocking syscall blocks all threads in it	Kernel threads don't have this issue, at the cost of heavier context switch
68	B	Thrashing = system busier paging than computing	Not a scheduling or disk algorithm term
69	B	RAID 1 = mirroring (full redundant copy)	RAID 0 is striping with no redundancy
70	C	Contiguous allocation leaves gaps between used regions as files grow/shrink = external fragmentation	Linked/indexed allocation avoid this by not requiring contiguous blocks
71	B	Classic waterfall: strictly sequential, no iteration back	Spiral/Agile are iterative by design
72	B	Spiral = repeated risk-driven prototyping cycles	Waterfall has no repeated cycles
73	B	Cyclomatic complexity (McCabe) measures independent paths through a control flow graph	Not about code length or team
74	B	Black-box testing = behaviour only, no internal code knowledge	White-box explicitly uses code structure
75	A	Verification = "building it right" (per spec); Validation = "building the right thing" (per user need)	B describes validation, the paired but different term
76	D	Unit, Integration, System (and Acceptance) are standard levels; "Compilation testing" is not a recognised testing level	—
77	B	COCOMO = COConstructive COst MOdel, for effort/cost estimation	Not a DB or network model
78	B	Functional requirement = what the system does	A describes a non-functional (performance) requirement
79	B	HAVING filters aggregated groups; WHERE filters rows before grouping	WHERE cannot reference aggregate functions directly
80	B	2NF: no partial dependency on part of a composite key	Transitive dependency is the 3NF condition
81	B	<code>SELECT DISTINCT</code> removes duplicate rows	<code>UNIQUE</code> is a constraint, not a query keyword for this purpose
82	B	FK enforces that referenced values exist (or are null) — referential integrity	Not about uniqueness within its own table
83	B	UPDATE modifies existing row values	INSERT adds new rows; ALTER changes schema
84	B	Candidate key = minimal unique-identifying attribute set	Not arbitrary, and not necessarily the chosen primary key
85	B	LEFT OUTER JOIN keeps all left rows, NULLs for unmatched right side	RIGHT OUTER JOIN is the mirror case

86	C	Isolation = concurrent transactions don't see each other's partial/intermediate state	Atomicity is "all or nothing"; Durability is "survives crash"
87	B	Network layer (e.g. IP) handles routing across networks	Data Link handles same-network framing
88	B	TCP: connection-oriented, reliable, with ACK/retransmit	UDP is connectionless, faster, unreliable
89	B	/24 = 24 network bits = 255.255.255.0	/16 would be 255.255.0.0
90	B	Switch forwards frames by MAC address (Layer 2)	Hub is a dumb broadcaster; Router works at Layer 3 by IP
91	B	DNS = Domain Name System, resolves names to IPs	Not encryption or routing computation
92	C	192.168.0.0/16 is the Class C-range private block	10.x is Class A private; 172.16–172.31 is Class B private
93	B	TCP handshake: SYN → SYN-ACK → ACK	Order and combined SYN-ACK packet matter
94	C	Full mesh: every node directly connects to every other	Star routes through a central hub; ring is a closed loop
95	B	<code>action</code> attribute names the form's target URL	<code>method</code> says GET/POST, not destination
96	B	<code>\$_POST</code> holds POST-submitted form data	<code>\$_GET</code> is for query-string data
97	B	<code></code> = unordered list (bulleted)	<code></code> is ordered (numbered)
98	B	<code>===</code> checks value AND type (strict)	<code>==</code> is loose/type-juggling
99	B	GET data is appended to the URL as <code>?key=value</code> pairs	Not sent silently in a cookie
100	B	Session data lives server-side; only the session ID token sits in a client cookie	Not stored as plaintext client-side
101	B	<code>.php</code> extension triggers server-side PHP processing	<code>.html/.htm</code> are served as static text, ignoring any PHP markup inside
102	B	<code>padding</code> = space between content and border	<code>margin</code> = space OUTSIDE the border, between elements
103	B	<code>alt</code> = fallback text / accessibility description	Not for sizing or compression
104	C	404 = Not Found	500 = server error; 200 = success; 301 = redirect
105	B	Service has no UI, runs in background	Activity has UI; Fragment is not one of the four core components

106	B	<code>onStop()</code> fires when the activity is no longer visible at all	<code>onPause()</code> fires while partially visible/covered
107	B	48×48 dp is Android's minimum recommended touch target	iOS uses 44×44 pt, a different unit and OS
108	B	<code>AndroidManifest.xml</code> declares components, permissions, SDK versions	<code>build.gradle</code> is the build config file, a different purpose
109	B	Flutter (Dart) compiles to native ARM code with its own rendering engine (Skia)	Not JS-interpreted or WebView-based
110	B	Implicit intent declares an action; OS resolves the handler(s)	Explicit intent names the exact component/class
111	B	<code>.apk</code> = Android Package, the installable signed archive	<code>.jar/.exe/.dmg</code> are unrelated package formats
112	B	Room is Android's official SQLite abstraction layer (Jetpack)	Core Data is iOS's equivalent, not Android's
113	B	Modus Ponens: from P and (P→Q), infer Q	Not the same as denying the antecedent
114	B	Inductive learning generalises FROM specific examples TO a general rule	Opposite direction from deduction
115	B	PROLOG programs = facts + rules, resolved via unification/backtracking	Not imperative sequencing
116	B	EBL uses one example plus background domain theory to deductively generalise a rule	Not purely statistical, not unsupervised clustering
117	B	KNN: supervised, "lazy"/instance-based (no explicit training phase, decides at query time)	Not unsupervised, not RL
118	B	K-means minimises within-cluster sum of squared distances to centroids	Not a classification-margin or entropy measure
119	B	Bagging averages many models trained on bootstrap samples → reduces variance	Boosting (not bagging) targets bias via sequential correction
120	B	Boosting trains sequentially, weighting/focusing on prior errors	Bagging (not boosting) trains independently in parallel

Correction note on Q31: 00001010 (decimal 10). One's complement (invert all bits) = 11110101. Two's complement = one's complement + 1 = **11110110**. So the correct answer is **B) 11110110**, not A — this worked correction is left visible deliberately as a reminder to **always redo the arithmetic yourself** rather than trust a printed key blindly; verify every 2's-complement question by hand in the exam.

PART 2 — DESCRIPTIVE QUESTION BANK

Model-answer **skeletons only** — the bullet points an examiner is scanning for, not full prose.
Expand each into full sentences in your own practice attempts; in the exam, write the expansion, not the bullet list.

PAPER-I — Descriptive Bank

Five-mark skeletons (22 — pick any 8 of 10 offered on the day)

1. **State the essential properties of an algorithm.**
 - Finiteness, Definiteness, Input, Output, Effectiveness — one line each.
2. **Differentiate a priori and a posteriori analysis.**
 - A priori = theoretical, machine-independent, before running; a posteriori = empirical, after running, machine-dependent. One-line table.
3. **Define Big-O, Big-Omega and Big-Theta.**
 - Formal $\exists c, n_0$ definitions for each; state $\Theta \Leftrightarrow O$ and Ω both hold; one growth-rate example each.
4. **Explain best, worst and average case using linear search.**
 - Best $O(1)$ (first element); Worst $O(n)$ (last/absent); Average $O(n)$, $(n+1)/2$ comparisons.
5. **Differentiate merge sort and quick sort (space, stability).**
 - Merge: $O(n)$ aux, stable. Quick: $O(\log n)$ aux average, not stable. One-line reason each.
6. **What is the Master Theorem? State its three cases.**
 - Form $T(n) = aT(n/b) + f(n)$; compare $f(n)$ to $n^{\log_b a}$; Case 1/2/3 result each.
7. **Define P, NP, NP-hard, NP-complete.**
 - P solved poly time; NP verified poly time; NP-hard at least as hard as NP; NP-complete = in NP and NP-hard.
8. **Explain the RAM model of computation.**
 - Single processor, unit-cost basic ops, no memory hierarchy/concurrency; why it makes analysis machine-independent.
9. **What are symbol tables? Name two implementations.**
 - Store name-attribute pairs (identifiers, types, scope); implementations: linear list, hash table, BST.
10. **State Kirchhoff/Boolean laws — De Morgan's theorems.**
 - $(AB)' = A' + B'$; $(A + B)' = A'B'$; one truth-table row to justify.
11. **Explain 1's and 2's complement representation of negative numbers.**
 - 1's: invert bits, end-around carry. 2's: invert + 1, no such carry adjustment; 2's complement is standard in modern hardware.
12. **What is a K-map? What problem does it solve?**
 - Graphical tool to minimise SOP/POS Boolean expressions by grouping adjacent 1s (or 0s) in powers of two.

13. Differentiate combinational and sequential circuits.

- Combinational: output depends only on current inputs, no memory. Sequential: output depends on inputs + present state (has memory/clock).

14. What is pipelining? State one hazard type.

- Overlap fetch/decode/execute of successive instructions; hazards: structural, data, control — describe one.

15. Differentiate direct and associative cache mapping.

- Direct: fixed one-to-one block-to-line mapping, fast but conflict-prone. Associative: any block anywhere, flexible but costlier hardware.

16. What is DMA? Why is it used?

- Direct Memory Access transfers data to/from memory without CPU handling every word — frees CPU for other work during I/O.

17. Differentiate a compiler and an interpreter.

- Compiler translates whole program to machine code before execution; interpreter executes line-by-line at runtime, no standalone executable produced.

18. What is a macro? How is it different from a function?

- Macro expanded inline at compile/assembly time (no call overhead, code duplicated); function called at runtime (call/return overhead, code shared).

19. State the phases of a compiler in order.

- Lexical → Syntax → Semantic → Intermediate code → Optimisation → Code generation.

20. What factors constrain mobile application development?

- Battery, network, screen size/density, input method (touch), fragmentation, interruptions — name and justify five.

21. Differentiate native and hybrid mobile applications.

- Native: platform language, best performance, per-OS codebase. Hybrid: one shared codebase (React Native/Flutter), near-native performance.

22. What is text-to-speech? Name its stages.

- Text analysis/normalisation → linguistic analysis (prosody) → waveform synthesis (concatenative/parametric/neural).

Ten-mark skeletons (14 — pick any 3 of 5 offered on the day)

1. Solve $T(n) = 2T(n/2) + n$ by (a) recursion tree (b) Master theorem.

- Tree: n work per level $\times \log n$ levels = $\Theta(n \log n)$. Master: $a=2, b=2, f(n)=n, e=1$, Case 2 $\rightarrow \Theta(n \log n)$. Show both derivations fully.

2. Explain the working of Quick Sort. State its average and worst-case complexity. (2025 exact question)

- Partition step (Lomuto/Hoare) with worked trace on a small array; average $\Theta(n \log n)$ balanced splits; worst $\Theta(n^2)$ on sorted input with poor pivot; fixes: randomised/median-of-three pivot.

3. Solve 0/1 Knapsack by dynamic programming for a given instance.

- Build the DP table $dp[i][w]$; recurrence $dp[i][w] = \max(dp[i-1][w], v_i + dp[i-1][w-w_i])$; fill for the given items/capacity; read optimal value from the final cell; trace back selected items.

4. Explain Prim's or Kruskal's algorithm for Minimum Spanning Tree with a worked example.

- State greedy criterion (Kruskal: sort edges ascending, add if no cycle via union-find; Prim: grow tree, always add cheapest edge crossing the frontier); trace on a 5–6 node graph; total MST weight.

5. Explain P vs NP vs NP-complete vs NP-hard with a diagram and one example each.

- Venn-diagram description: $P \subseteq NP$; NP-complete = $NP \cap NP\text{-hard}$; example NP-complete: SAT/TSP-decision; state Cook's theorem names SAT as the first proven NP-complete problem.

6. Design and simplify a combinational circuit for a given Boolean function using K-map.

- Draw truth table $\rightarrow$ K-map $\rightarrow$ group adjacent 1s in largest power-of-2 blocks $\rightarrow$ read off minimal SOP $\rightarrow$ draw the gate-level circuit.

7. Explain the phases of a compiler with a diagram, using a sample statement.

- Trace `a = b + c * d` through lexical (tokens) $\rightarrow$ syntax (parse tree) $\rightarrow$ semantic (type check) $\rightarrow$ intermediate code (3-address code) $\rightarrow$ optimisation $\rightarrow$ target code.

8. Explain the Android activity lifecycle with a diagram.

- Draw the state diagram (`onCreate` $\rightarrow$ `onStart` $\rightarrow$ `onResume` $\rightarrow$ `onPause` $\rightarrow$ `onStop` $\rightarrow$ `onDestroy`, plus `onRestart` loop); explain why it exists (interruptions, OS memory reclamation).

9. What are the four Android components? Explain each with an example.

- Activity (UI screen), Service (background task, e.g. music player), Broadcast Receiver (system event listener), Content Provider (shared structured data access).

10. Differentiate explicit and implicit intents; give one example each.

- Explicit: names target class, e.g. launching your own `SettingsActivity`. Implicit: declares action (`ACTION_SEND`), OS resolves candidate apps.

11. Discuss the frameworks and tools used in mobile app development. (2025 exact question)

- IDEs (Android Studio, Xcode); cross-platform (React Native, Flutter, Ionic); storage (SQLite/Room/Core Data); UI guidelines (Material/HIG); testing (Espresso/XCTest/Appium); crash reporting (Crashlytics/Sentry).

12. Explain synchronization and replication of mobile data.

- Define both terms; conflict scenarios (offline edit + server edit); resolution strategies (last-write-wins, manual merge); offline-first design principle.

13. Explain the packaging and deployment process for an Android app.

- Build $\rightarrow$ package into APK/AAB $\rightarrow$ sign with private key $\rightarrow$ version (`versionCode/versionName`) $\rightarrow$ upload to Play Console $\rightarrow$ review $\rightarrow$ staged rollout.

14. **Explain how a floating-point number is represented (IEEE-754 single precision).**

- Sign bit, 8-bit exponent (biased by 127), 23-bit mantissa; worked example converting a small decimal to its bit pattern.

Fifteen-mark skeletons (8 — pick any 2 of 4 offered on the day)

1. **Describe and compare divide-and-conquer and greedy paradigms, with a concrete example of each.** (2025 exact question)
 - D&C: divide/conquer/combine steps; example merge sort, recurrence + Master theorem solve. Greedy: candidate/selection/feasibility/objective functions; example fractional knapsack, worked table. Comparison table: where the work sits, correctness guarantee (exchange argument vs recursive correctness), when each applies (independent subproblems vs greedy-choice + optimal substructure).
2. **Explain dynamic programming with two fully worked examples (0/1 Knapsack and LCS).**
 - Define overlapping subproblems + optimal substructure; DP table for knapsack (from Paper-I bank above); DP table for LCS with traceback reading off the actual subsequence; contrast with D&C (no overlap) and greedy (no correctness guarantee here).
3. **Design a MOD-10 (BCD) synchronous counter. Draw the state diagram, state table and logic circuit.**
 - 4 flip-flops needed ($\lceil \log_2 10 \rceil = 4$); state diagram $0000 \rightarrow \dots \rightarrow 1001 \rightarrow 0000$ (10 states, skip 1010–1111); state table with next-state and flip-flop input equations (derive JK/D inputs via K-map); draw the resulting circuit.
4. **Explain the complete compiler pipeline with a running example, including code optimisation.**
 - Full pipeline diagram; trace one small program through lexical/syntax/semantic/intermediate/optimisation/codegen; name at least two optimisation techniques (constant folding, dead code elimination, loop-invariant code motion) with a before/after snippet.
5. **Discuss the challenges in designing the right UI for mobile applications.** (2025 exact question)
 - Screen fragmentation $\rightarrow$ responsive/dp-based layout; touch imprecision $\rightarrow$ 48dp targets; platform convention $\rightarrow$ Material/HIG; interruption-prone context $\rightarrow$ short tasks, high contrast, thumb-zone reachability; close with the one-line thesis (obvious/reachable/reversible next action).
6. **Explain CPU addressing modes with an example instruction for each, and their trade-offs.**
 - Immediate, direct, indirect, register, register-indirect, indexed, base-register — one example instruction and one trade-off (speed vs flexibility) per mode.
7. **Explain Strassen's matrix multiplication algorithm; derive and compare its complexity with the conventional method.**
 - Naive $\Theta(n^3)$; block-decompose into 8 vs Strassen's 7 products; write all 7 P-terms and the 4 C-block reconstructions; Master theorem derivation to $\Theta(n^{2.807})$; honest limitations (constant factor, additions overhead, crossover point).
8. **Discuss NP-completeness, Cook's theorem, and reduction technique, with SAT as example.**
 - Define P/NP/NP-hard/NP-complete; state Cook-Levin theorem (SAT is NP-complete); explain polynomial-time reduction as the proof technique for showing a new problem is NP-complete (reduce a known NP-complete problem to it); significance of P=NP question.

PAPER-II — Descriptive Bank

Five-mark skeletons (22 — pick any 8 of 10 offered on the day)

1. Differentiate multiprogramming and multitasking.

- Multiprogramming: several jobs resident in memory, CPU switches on I/O wait, no user-interaction focus. Multitasking: rapid switching giving illusion of simultaneous execution, typically interactive/time-shared.

2. State the necessary conditions for deadlock.

- Mutual exclusion, Hold-and-wait, No preemption, Circular wait — all four must hold simultaneously.

3. Differentiate paging and segmentation.

- Paging: fixed-size physical blocks, no external fragmentation, may have internal fragmentation. Segmentation: variable-size logical units, may have external fragmentation, matches program's logical structure.

4. What is Belady's anomaly?

- Under FIFO, page faults can INCREASE when the number of frames increases — counter to intuition.

5. Differentiate verification and validation.

- Verification: are we building it right (per spec)? Validation: are we building the right thing (per user needs)?

6. What is cyclomatic complexity? State its formula.

- $V(G) = E - N + 2P$ (edges, nodes, connected components); measures independent linear paths through control flow.

7. Differentiate black-box and white-box testing.

- Black-box: behaviour only, no code knowledge, input/output based. White-box: internal code structure known, tests specific paths/branches.

8. What is normalization? Define 1NF.

- Process of organising data to reduce redundancy/anomalies; 1NF: atomic values only, no repeating groups.

9. Differentiate DELETE, TRUNCATE and DROP in SQL.

- DELETE: removes rows, can be conditional, logged, rollback-able. TRUNCATE: removes all rows fast, minimal logging. DROP: removes the entire table structure.

10. What is a foreign key? What integrity does it enforce?

- Column(s) referencing a primary/unique key of another table; enforces referential integrity (no orphaned references).

11. Differentiate LAN, MAN and WAN.

- LAN: small area (building/campus). MAN: city-scale. WAN: country/global scale, often uses leased/public infrastructure.

12. State the layers of the OSI model in order.

- Physical, Data Link, Network, Transport, Session, Presentation, Application (bottom to top).

13. Differentiate TCP and UDP.

- TCP: connection-oriented, reliable, ordered, higher overhead. UDP: connectionless, unreliable, low overhead, used for streaming/DNS.

14. What is DNS? Why is it needed?

- Domain Name System translates human-readable domain names to IP addresses; needed because

humans use names, routers use numeric addresses.

15. Explain the basic HTML document structure.

- `<!DOCTYPE html>`, `<html>` root, `<head>` (metadata/title), `<body>` (visible content) — name each with one purpose.

16. Differentiate GET and POST methods.

- GET: data in URL query string, visible, size-limited, idempotent. POST: data in request body, hidden, larger limits, used for state-changing submissions.

17. What are PHP sessions? Where is session data stored?

- Mechanism to persist data across requests for one user; data stored server-side, only the session ID travels via a client cookie.

18. Differentiate client-side and server-side scripting.

- Client-side (JavaScript): runs in browser, no DB access. Server-side (PHP): runs on server, generates HTML, has DB access, source never sent to client.

19. Explain the CSS box model.

- Content → Padding → Border → Margin, innermost to outermost; briefly describe each layer's role.

20. State the phases of website development.

- Planning, Design, Implementation (front-end + back-end), Testing, Deployment, Maintenance — the syllabus stresses Implementation/Testing/Maintenance.

21. What is inductive learning? Give an example.

- Generalising a rule from specific labelled examples; example: learning "all observed swans are white" from a set of swan sightings.

22. Differentiate supervised and unsupervised learning.

- Supervised: labelled data, learns input→output mapping (e.g. KNN, regression). Unsupervised: unlabelled data, finds structure (e.g. K-means clustering).

Ten-mark skeletons (14 — pick any 3 of 5 offered on the day)

1. Explain any two CPU scheduling algorithms with a Gantt chart and average waiting time calculation.

- Pick FCFS and SJF (or Round Robin); given arrival/burst times, draw Gantt chart, compute completion/turnaround/waiting times, average them; note starvation/context-switch trade-offs.

2. Explain the necessary conditions for deadlock and one deadlock-prevention strategy for each.

- List all four conditions; for each, name a concrete OS technique to violate it (e.g. resource ordering breaks circular wait; requesting all resources upfront breaks hold-and-wait).

3. Explain paging with address translation, using a numeric example.

- Logical address split into page number + offset; page table maps page→frame; worked example translating a given logical address to a physical address.
- 4. **Compare FIFO, LRU and Optimal page replacement algorithms on a given reference string.**
 - Trace all three algorithms on the same reference string with a given number of frames; count page faults for each; conclude $\text{Optimal} \leq \text{LRU} \leq \text{FIFO}$ (in general).
- 5. **Explain the Waterfall and Spiral SDLC models, and compare them.**
 - Waterfall: sequential phases, diagram; Spiral: repeated risk-driven cycles, diagram; comparison table (flexibility, risk handling, when each suits a project).
- 6. **Explain the E-R model with a diagram, converting it to a relational schema.**
 - Entities, attributes, relationships (1:1, 1:M, M:N) in an E-R diagram for a sample scenario (e.g. Student-Course); convert to tables with PK/FK, noting how M:N needs a junction table.
- 7. **Write SQL to create a table, insert records, and answer 2–3 queries (JOIN, GROUP BY, subquery).**
 - `CREATE TABLE` with constraints; `INSERT` sample rows; queries using `JOIN`, `GROUP BY ... HAVING`, and a subquery; show expected output rows for each.
- 8. **Explain the OSI reference model layer by layer with the function/protocol example of each.**
 - Seven layers, one function + one protocol/example per layer (e.g. Network: IP; Transport: TCP/UDP; Application: HTTP/FTP).
- 9. **Differentiate and explain the working of a router, switch and hub.**
 - Hub: physical-layer broadcaster, no intelligence. Switch: data-link layer, MAC-address-based forwarding, builds a MAC table. Router: network-layer, IP-based routing between different networks.
- 10. **Explain the difference between GET and POST methods in HTML forms. (2025 exact question)**
 - Full comparison table (location of data, superglobal, visibility, size limit, idempotency, suitable use cases); short PHP code snippet for each showing `$_GET/$_POST` retrieval.
- 11. **What are the fundamental syntax rules and use of variables in PHP scripting? (2025 exact question)**
 - `$` prefix, no type declaration, dynamic typing, case sensitivity, `.` for concatenation, embedding via `<?php ?>`, `.php` extension requirement; short worked code example with traced output.
- 12. **Write a PHP script to connect to MySQL and display records from a table; explain each step.**
 - `mysqli_connect()` (host, user, pass, db) → check connection → `mysqli_query()` → `mysqli_fetch_assoc()` loop → `mysqli_close()`; trace expected output for 2–3 sample rows.
- 13. **Explain inductive learning and explanation-based learning, contrasting the two.**
 - Inductive: generalise from many examples statistically/empirically. EBL: generalise from one example using existing domain theory, deductively; state which needs more prior knowledge vs more data.
- 14. **Explain K-means clustering with a worked numeric example (2–3 iterations).**
 - Choose k initial centroids; assign points to nearest centroid; recompute centroids; repeat for 2 iterations on a small 2D point set; show convergence / stopping condition.

Fifteen-mark skeletons (8 — pick any 2 of 4 offered on the day)

1. **Explain process scheduling, and derive average waiting/turnaround time for FCFS, SJF and Round Robin on the same data set.**
 - One data set (arrival + burst times) run through all three algorithms; three Gantt charts; comparison table of averages; conclude with trade-off discussion (fairness vs throughput vs starvation).
2. **Explain memory management techniques: paging, segmentation, and virtual memory with demand paging.**
 - Paging (fixed frames, page table, internal fragmentation); Segmentation (logical variable-size units, external fragmentation); Virtual memory/demand paging (pages loaded on demand, page fault handling, thrashing risk); one diagram per technique.
3. **Discuss the SDLC in detail: waterfall, spiral, and agile/prototype models, with when to use each.**
 - Diagram + description for each of the three-plus models; a decision guide (well-understood requirements → waterfall; high risk/uncertainty → spiral; changing requirements → agile).
4. **Explain normalization up to BCNF with a worked example table taken through each normal form.**
 - Start with an unnormalised table; show partial dependency removed (1NF→2NF); transitive dependency removed (2NF→3NF); BCNF condition (every determinant is a candidate key) checked and fixed if violated; show the final decomposed schema.
5. **Explain the OSI and TCP/IP models, mapping layers between them, with one protocol example per layer.**
 - Seven OSI layers vs four/five TCP/IP layers, mapping table; one protocol example named per layer; explain why TCP/IP became the practical standard despite OSI being the formal reference model.
6. **Describe the phases of website development, from planning to maintenance.** *(2025 exact question)*
 - Planning/requirements → Design (responsive, interactive states) → Implementation (front-end HTML/CSS/JS, back-end PHP+MySQL/Node/Django) → Testing (functional, cross-browser, security) → Deployment → Maintenance (continuous, unlike shrink-wrapped software); state the "never finished" thesis explicitly.
7. **Explain supervised, unsupervised and ensemble learning methods (KNN, K-means, Bagging/Boosting) with one example each.**
 - KNN: instance-based classification example. K-means: clustering example with centroid iteration. Bagging vs Boosting: variance vs bias reduction, one real algorithm named each (Random Forest for bagging, AdaBoost for boosting).
8. **Explain database transactions, the ACID properties, and concurrency control techniques.**
 - Define Atomicity, Consistency, Isolation, Durability with a one-line example failure each property prevents; concurrency control: locking (shared/exclusive), timestamp ordering; brief mention of deadlock in the DB context (parallels OS deadlock).

Cross-check against 2025 papers — calibration note

Every question marked *(2025 exact question)* above is a direct match — in topic and mark value — to a question that actually appeared in the CTSE 2025 CS-Degree Paper-I or Paper-II, per

[_raw/existing_txt/CTSE 2025 - Paper I - Questions and Answers.txt](#) and

[CTSE 2025 - Paper II - Questions and Answers.txt](#). If you can produce all of those answers from a blank page, at the stated mark's depth, you have matched the 2025 candidate's preparation on the exact points that were tested last year — treat everything else in this bank as the reasonable extension of that same syllabus coverage into adjacent, equally-likely topics.